Machine Learning

Samatrix Consulting Pvt Ltd

Decision Tree

Decision Tree

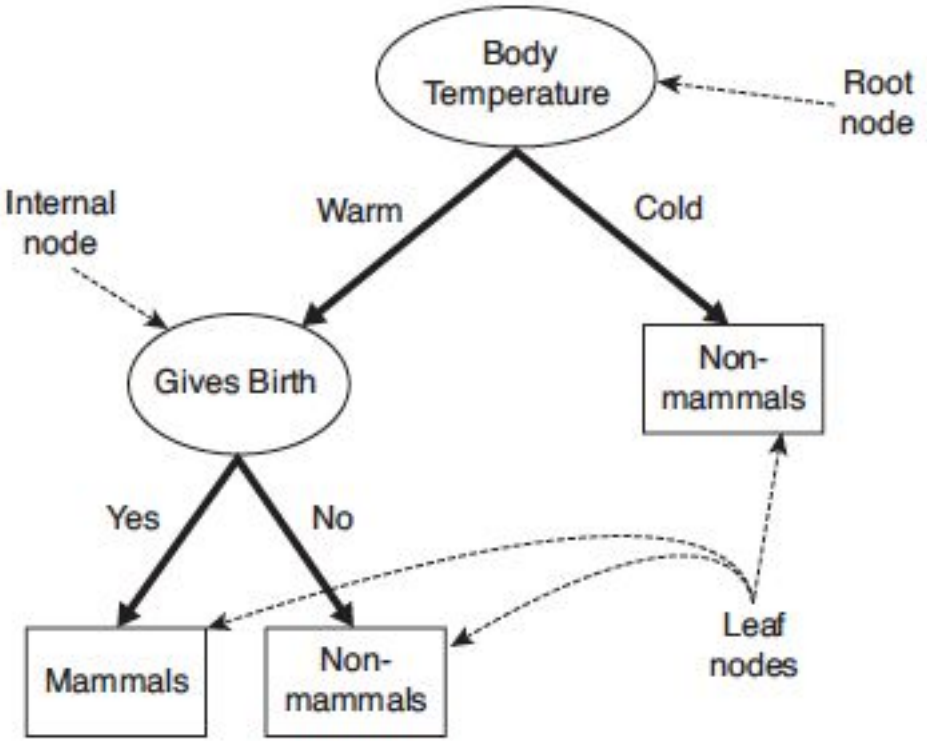
- This chapter covers the tree-based methods for regression and classification.
- The tree-based methods **stratify** or **segment** the predictor space into a number of simple regions.
- We use the mean or the mode of the training observation to make the prediction.
- These methods are also known as **decision tree** methods because they use the set of splitting rules to segment the predictor space, which can be summarized in a tree.

Decision Tree

- The Tree-based methods are simple. They are very useful for interpretation.
- In this chapter we will also study about bagging, random forests, and boosting.
- These approaches use multiple trees that we can combine to get a single consensus prediction.

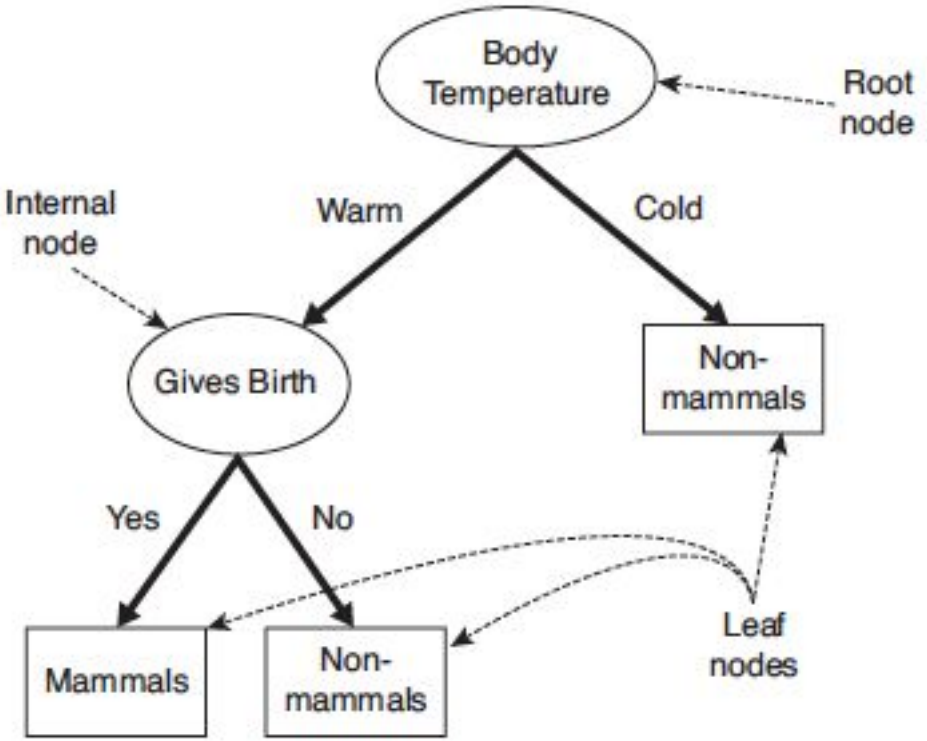
How a Decision Tree works

- Suppose the scientists discover a new species.
- We need to identify whether it is a mammal or a non-mammal.
- We may ask a series of questions about the characteristics of the species.
- The first question that we can ask is whether the species is cold blooded or warm blooded.
- If it is cold blooded, then it cannot be a mammal.
- Else it can either be a bird or a mammal.



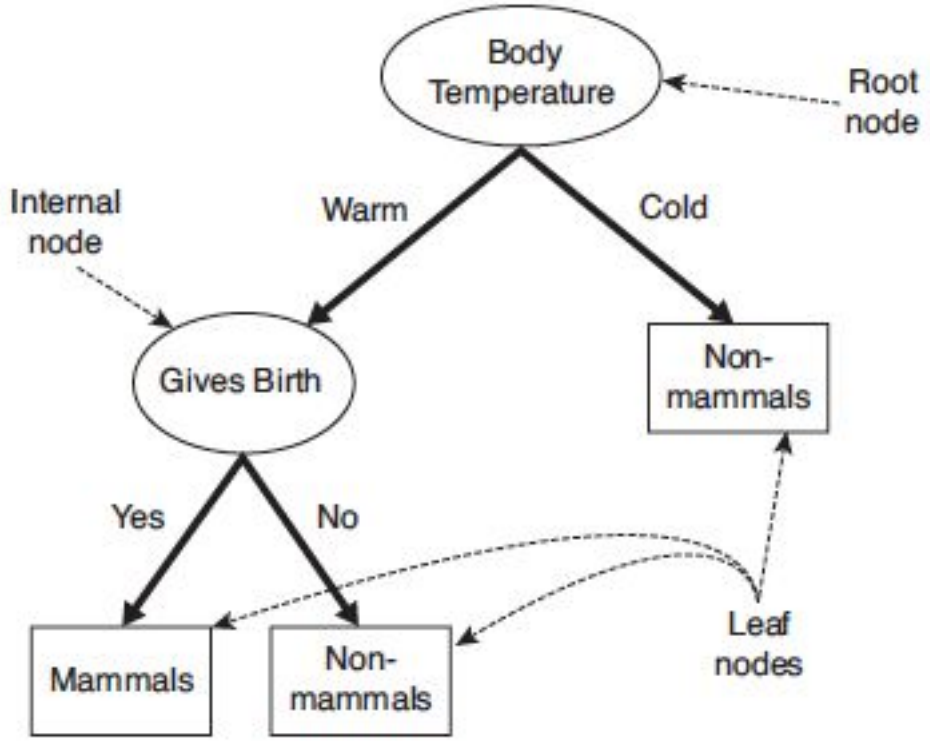
How a Decision Tree works

- In this case, we can ask a follow-up question.
- Whether the female of the species give birth to their young?
- If they give birth, the species definitely belongs to mammal.
- If they do not, they might be non-mammals (with an exception of egg-laying mammals such as platypus and spiny anteater).



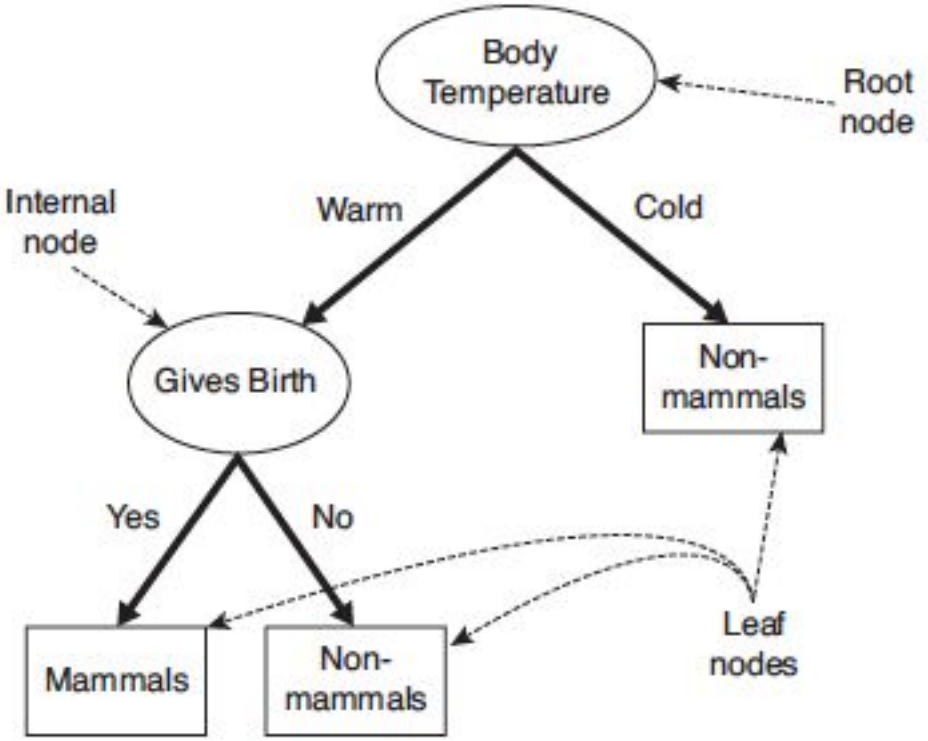
How a Decision Tree works

- We can solve such classification problems by asking a series of questions about the attribute of the test record.
- Every time, an answer is provided, we can ask a follow-up question until a class label of the record is reached.
- Such follow-up questions and their answers can be organized in the form for a decision tree, which is a hierarchical structure that contains nodes and directed edge.
- Figure-1 illustrates the decision tree for the mammals.



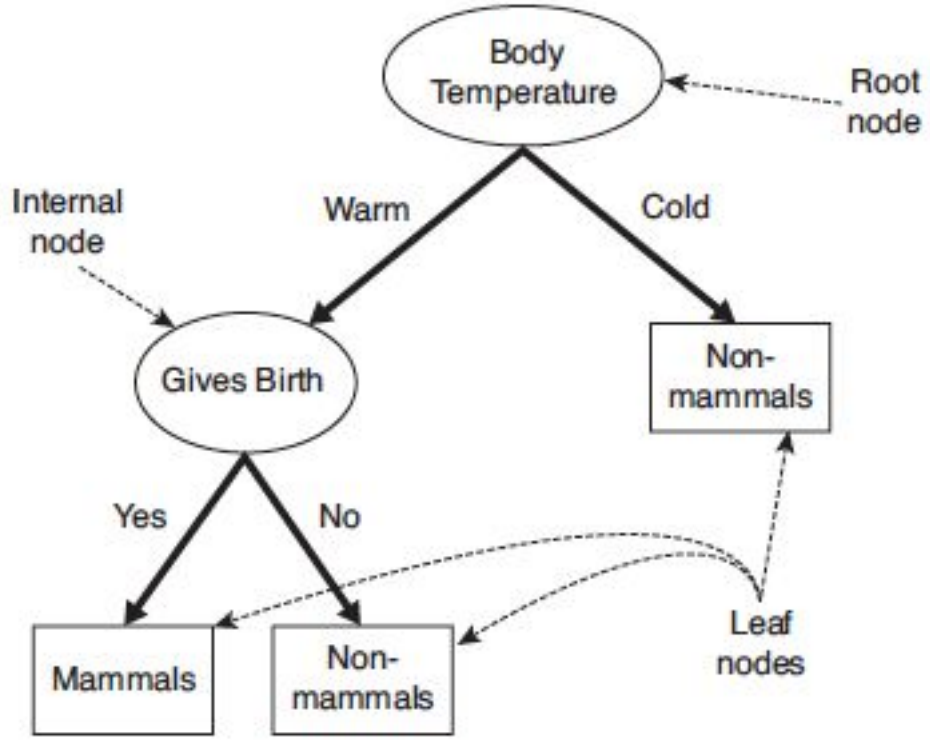
Structure of Decision Tree

- The tree has three types of nodes
  - A **root node** has no incoming edge and zero or more outgoing edges.
  - **Internal node** has exactly one incoming edge and two or more outgoing edges.
  - **Leaf or terminal node** has exactly one incoming edge and no outgoing edges.



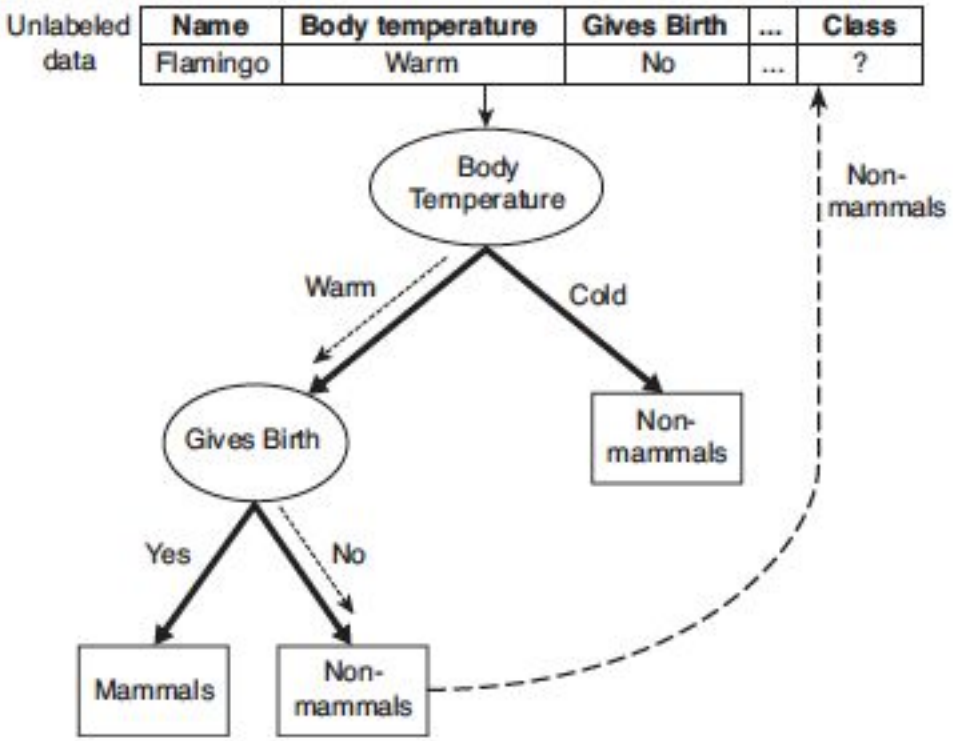
Structure of Decision Tree

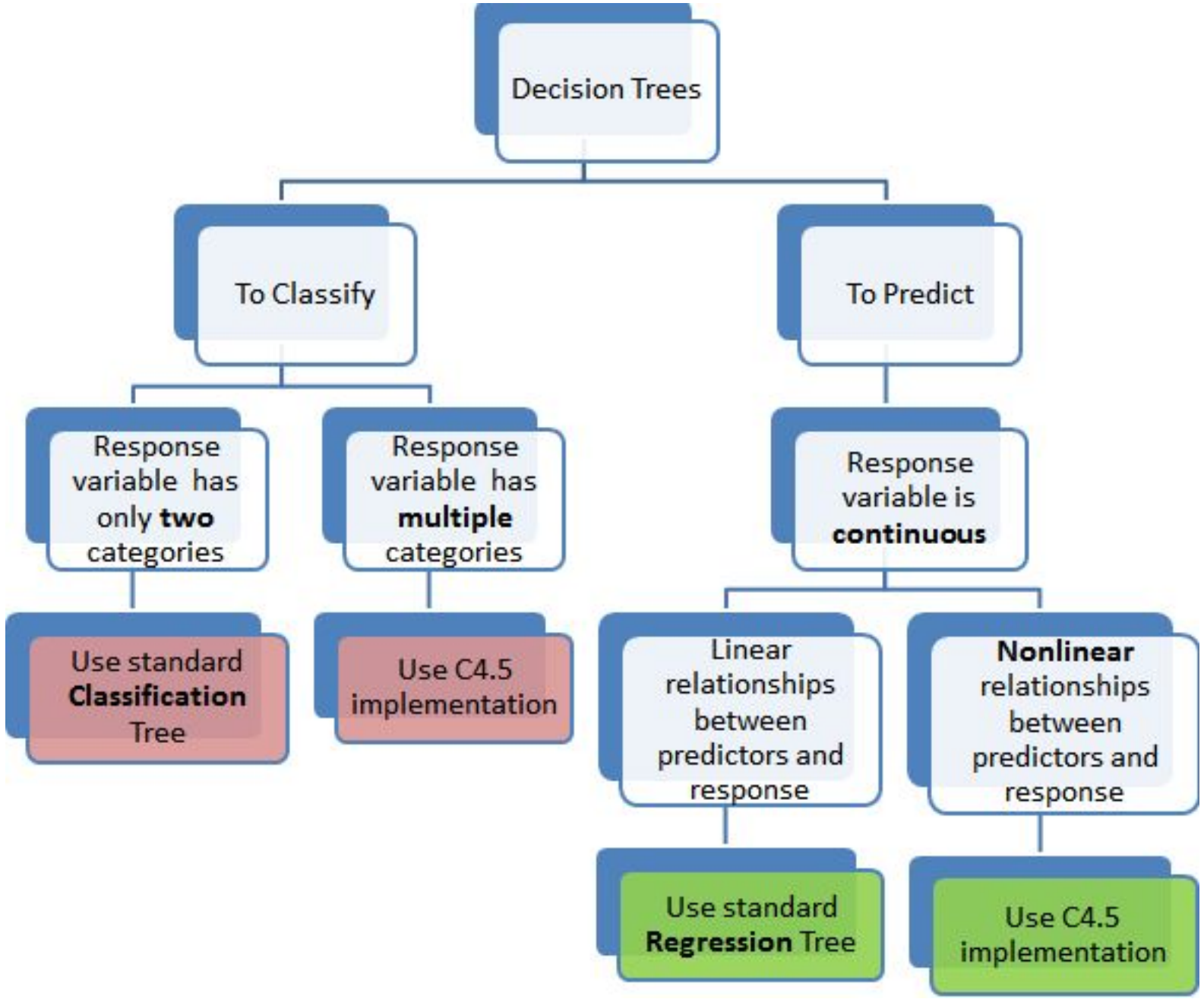
- We assign each leaf node a class label.
- The non-terminal nodes which include the root and internal nodes, contain attribute test conditions to separate records that have different characteristics.
- The root node as shown in Figure – 1 uses the attribute Body Temperature to separate warm-blooded from cold-blooded vertebrates.
- Since all cold-blooded vertebrates are non-mammals, a leaf node labeled Non-mammals has been created as the right child of the root node.
- If the vertebrate is warm-blooded, a subsequent attribute, Gives Birth is used to distinguish mammals from other warm-blooded creatures, which are mostly birds.

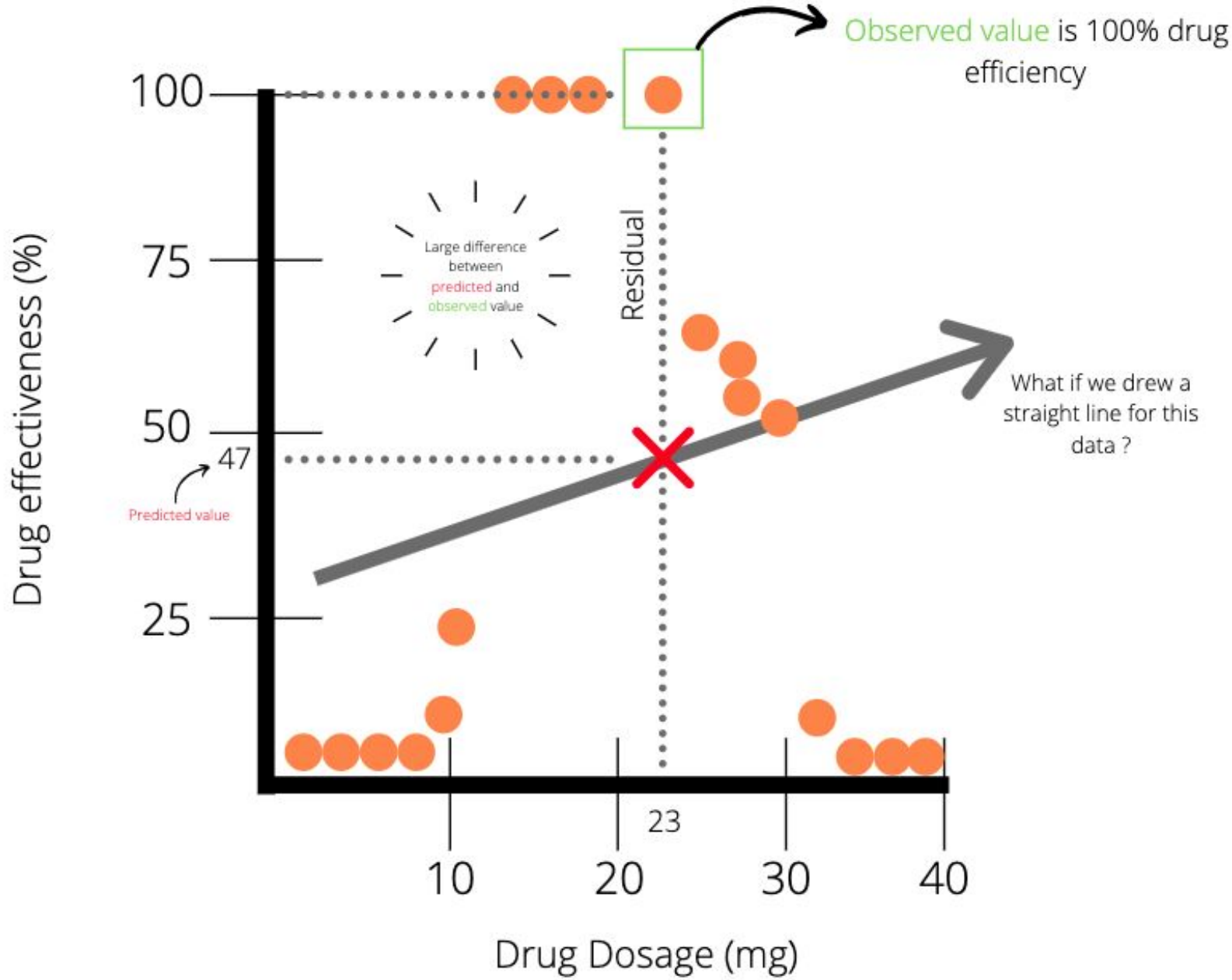


Structure of Decision Tree

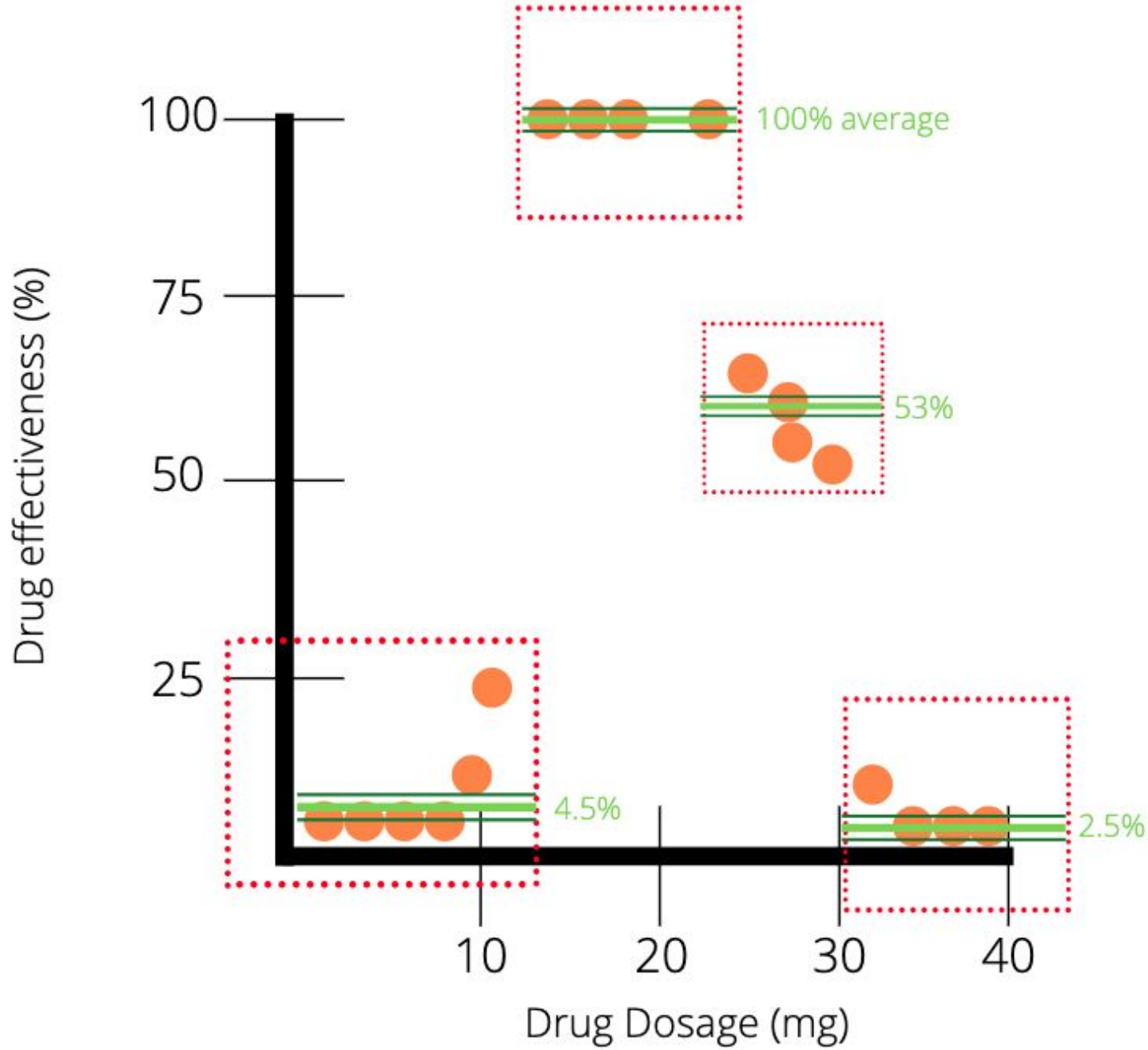
- Once a decision tree has been constructed, we can easily classify a test record.
- We can start from the root node, apply the test condition to the record, and follow the appropriate branch that is based on the outcome of the test.
- We will either land on another internal node on which we can apply a new test condition, or to a leaf node.
- Finally, we assign the class label that has been associated with the leaf node to the record.
- Figure – 2 illustrates the process to trace the path in the decision tree to predict the class label of a flamingo.
- The path terminates at a leaf node labeled Non-mammals.



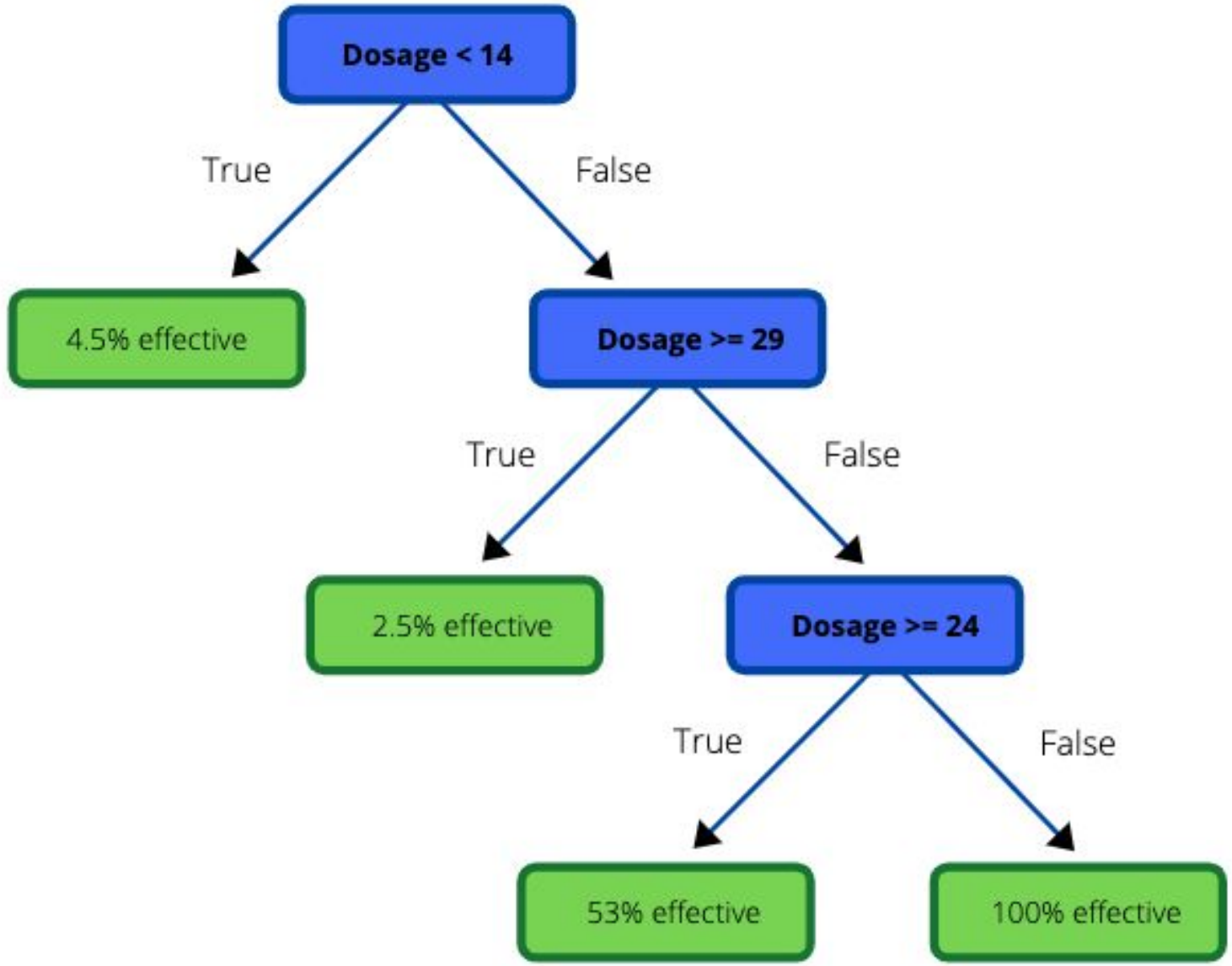

Decision Trees
To Classify
To Predict
Response variable has only **two** categories
Response variable has **multiple** categories
Response variable is **continuous**
Use standard **Classification** Tree
Use C4.5 implementation
Linear relationships between predictors and response
Nonlinear relationships between predictors and response
Use standard **Regression** Tree
Use C4.5 implementation


Observed value is 100% drug efficiency
100
75
50
25
Drug effectiveness (%)
Large difference between predicted and observed value
Residual
What if we drew a straight line for this data ?
47
Predicted value
23
10
20
30
40
Drug Dosage (mg)

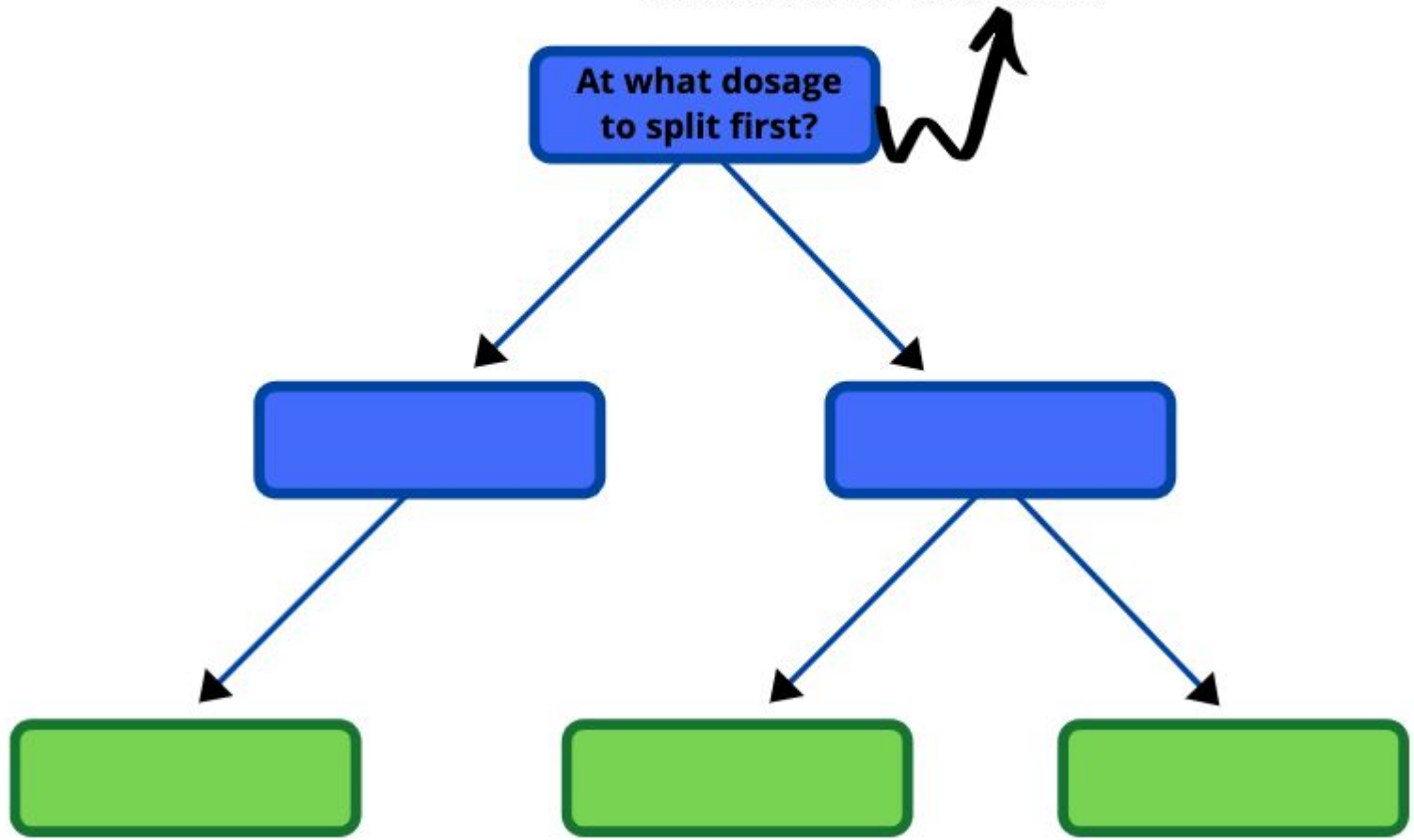
Samatrix.io

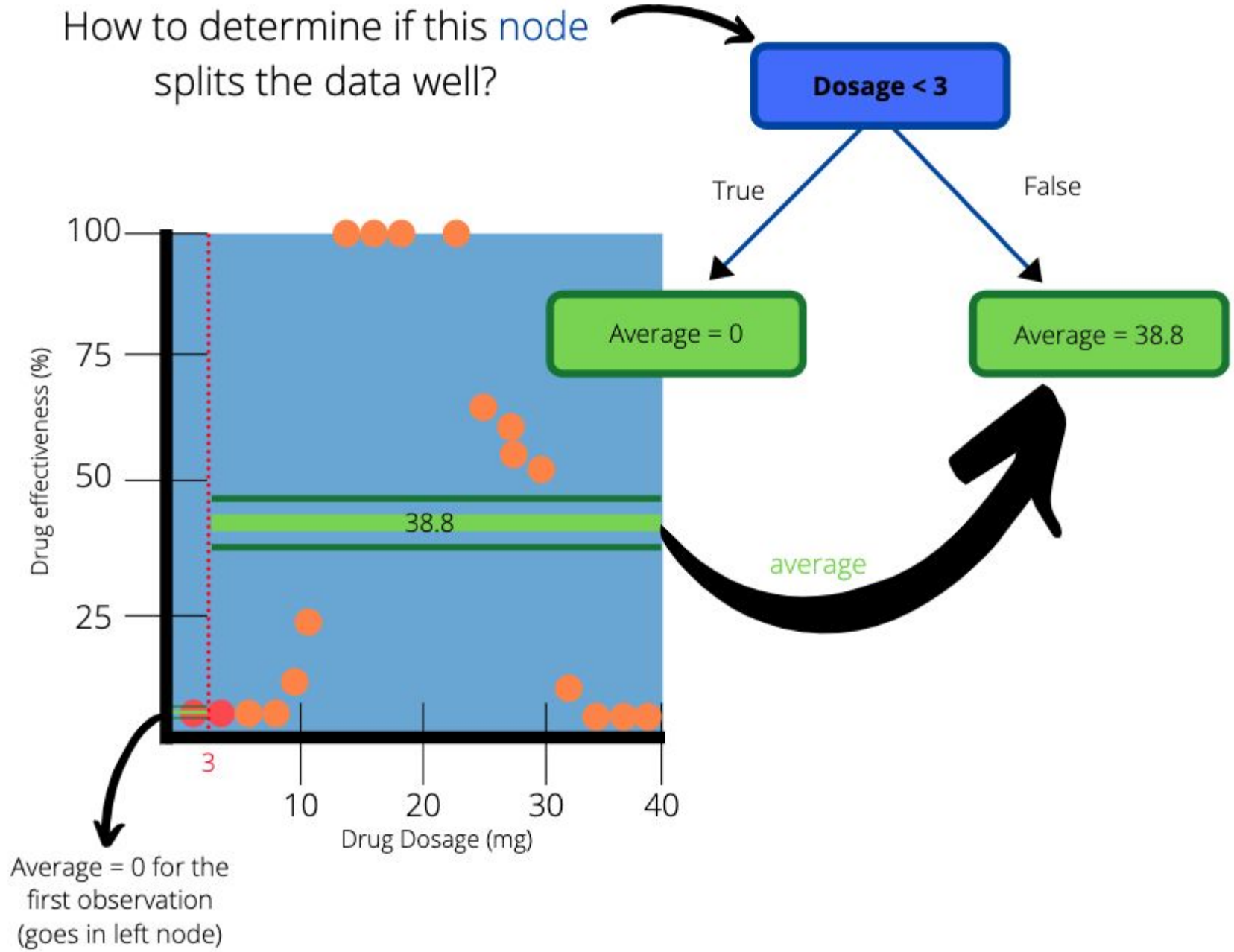

100
75
50
25
Drug effectiveness (%)
100% average
53%
4.5%
2.5%
10
20
30
40
Drug Dosage (mg)

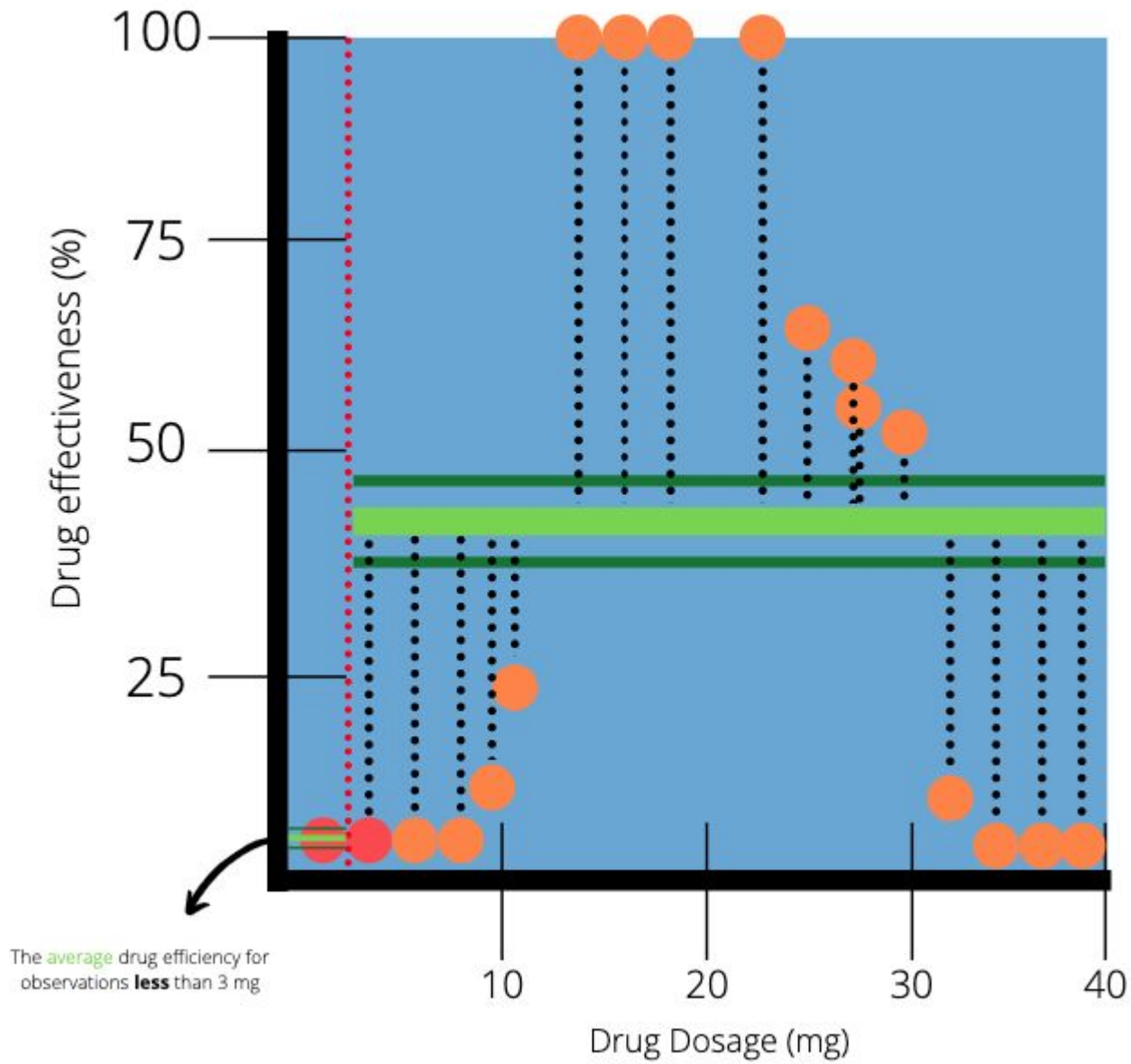
Samatrix.io


Dosage < 14
True
False
4.5% effective
Dosage >= 29
True
False
2.5% effective
Dosage >= 24
True
False
53% effective
100% effective

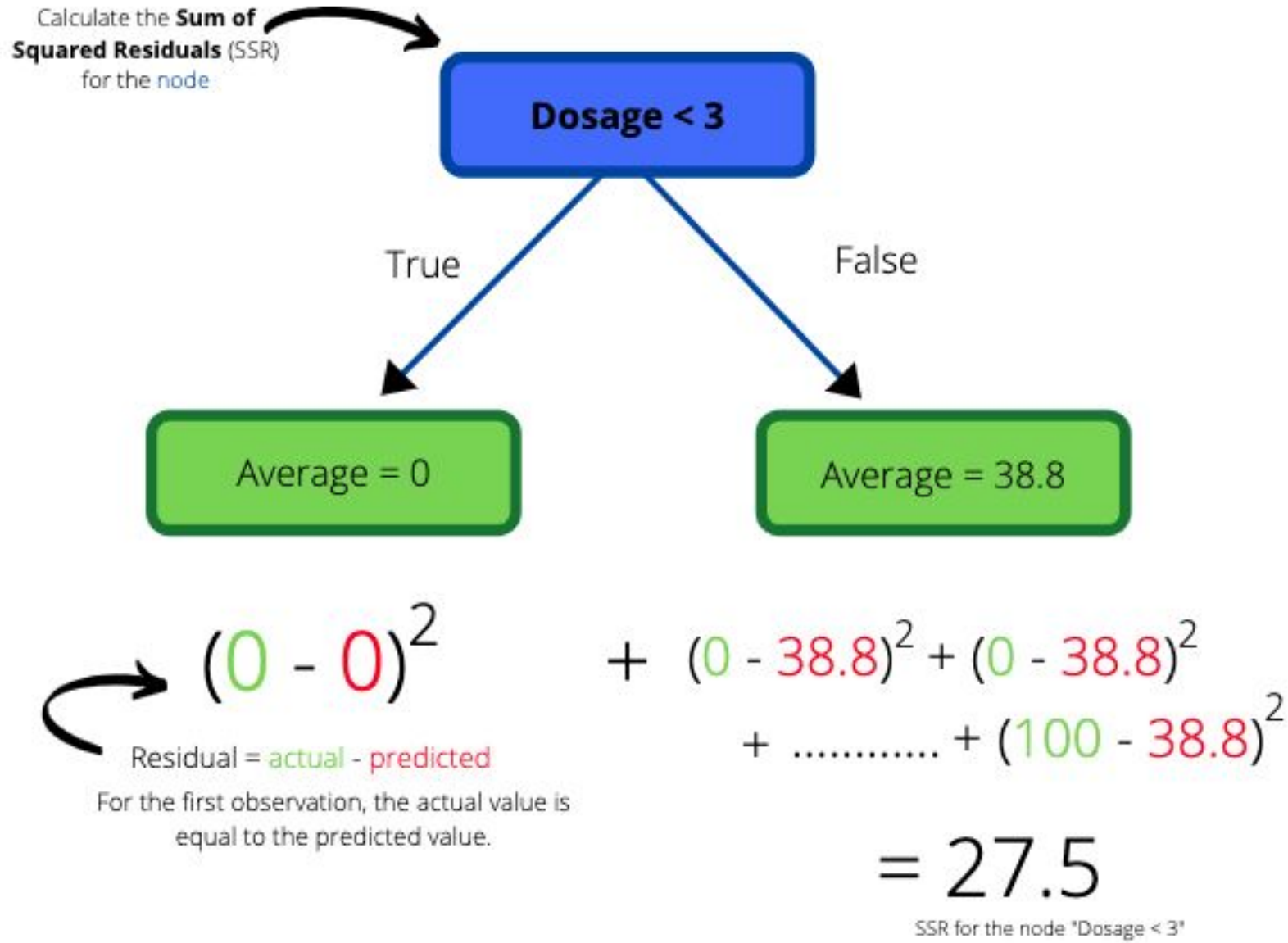
But remember our first question from classification decision trees ...Which attribute should be at the "**Root Node**"?

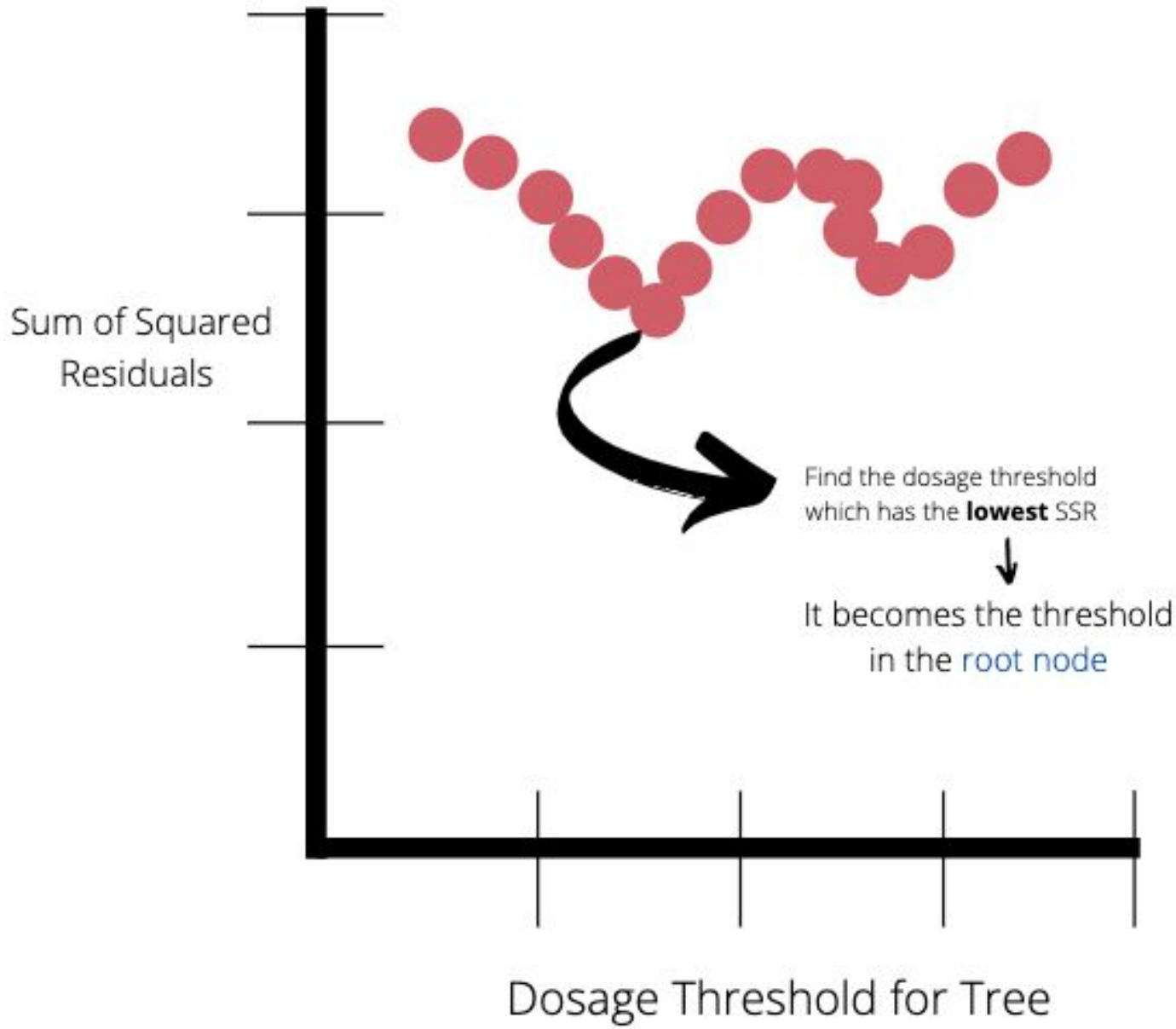



How to determine if this node splits the data well?
Dosage < 3
True
False
Average = 0
Average = 38.8
average
100
75
50
25
Drug effectiveness (%)
38.8
3
10
20
30
40
Drug Dosage (mg)
Average = 0 for the first observation (goes in left node)


100
75
50
25
Drug effectiveness (%)
The average drug efficiency for observations less than 3 mg
10
20
30
40
Drug Dosage (mg)

Samatrix.io


Calculate the **Sum of Squared Residuals** (SSR) for the node
Dosage < 3
True
False
Average = 0
Average = 38.8
$(0 - 0)^2$
Residual = actual - predicted
For the first observation, the actual value is equal to the predicted value.
$+ (0 - 38.8)^2 + (0 - 38.8)^2$
$+ \ldots\ldots\ldots + (100 - 38.8)^2$
$= 27.5$
SSR for the node "Dosage < 3"

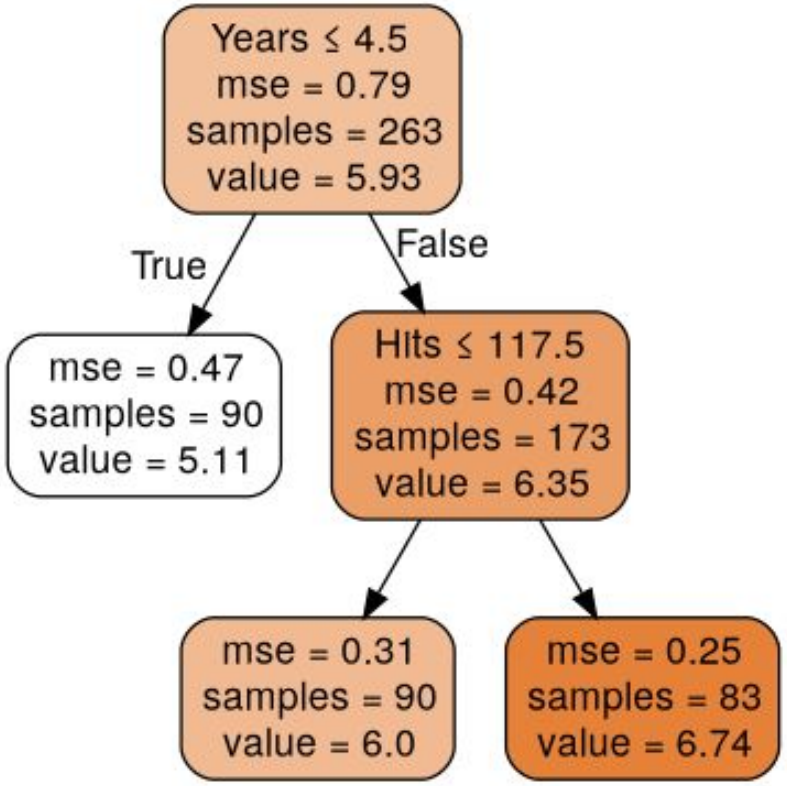

Sum of Squared Residuals
Find the dosage threshold which has the **lowest** SSR
It becomes the threshold in the root node
Dosage Threshold for Tree

Predict Baseball Player Salary Using Tree

- In the previous chapter, we used the Player data set to predict the baseball player's Salary based on Years (the number of years that he has played in major leagues) and Hits (the number of hits that he made in the previous year).
- We remove the observations that are missing the Salary values, and log-transform Salary so that its distribution has more of typical bell-shape.

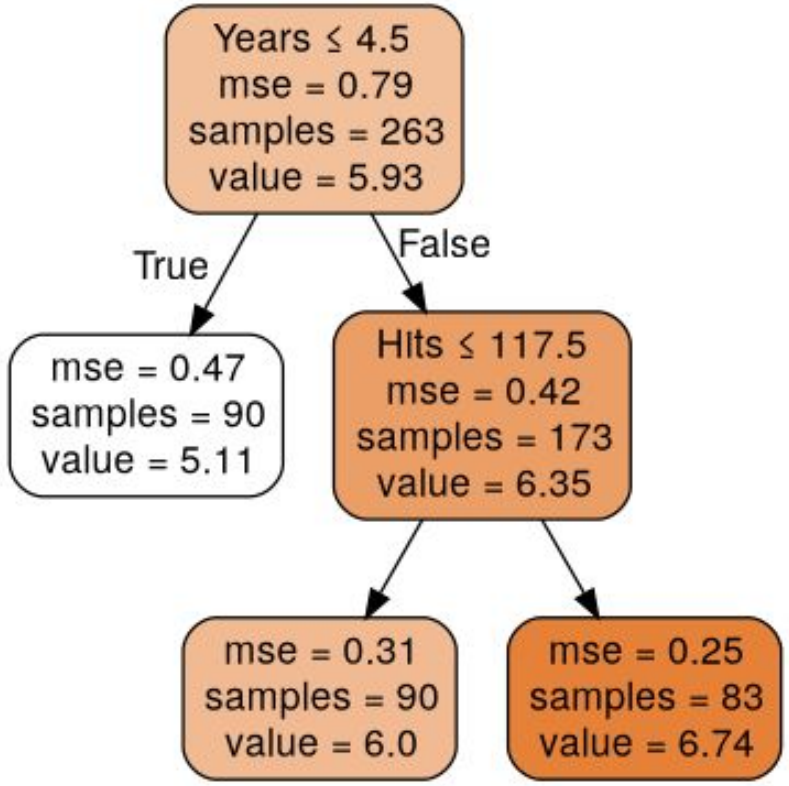
Predict Baseball Player Salary Using Tree

- We have shown the regression tree fit for this data.
- We have used a series of splitting rules that start at the top of the tree.
- The top split has assigned observations with Years < 4.5 to the left branch.
- We can predict the salary for these players by using the mean response value for the players with Years < 4.5. The mean log salary of such players 5.107.
- So, we can predict $1000 \times e^{5.107} = \$165{,}174$ for these players



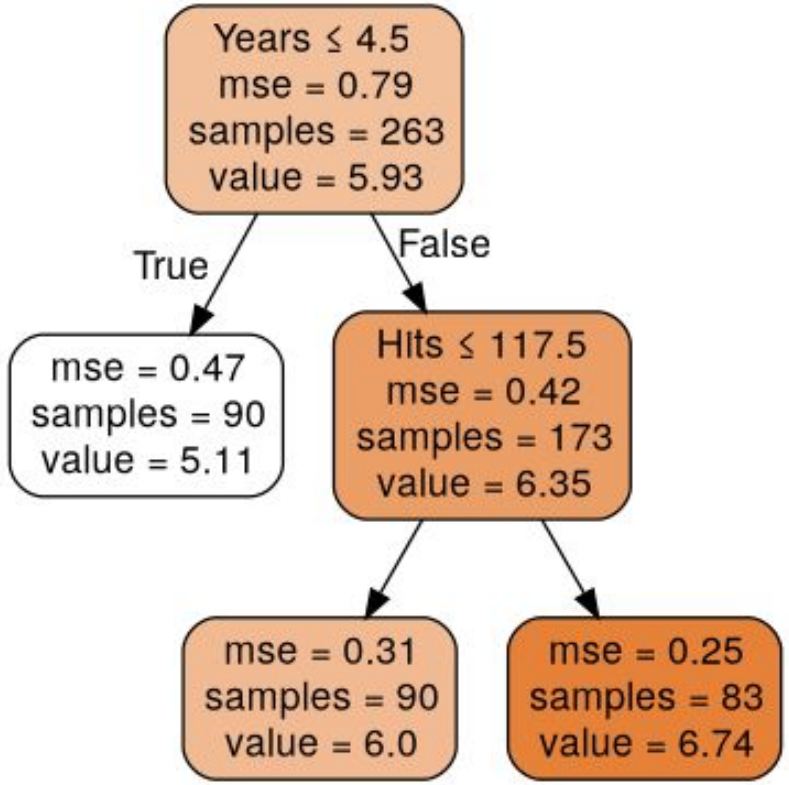
Predict Baseball Player Salary Using Tree

- We have assigned the players with Years >= 4.5 to the right-hand branch and subdivided the group by Hits.
- Now we have segmented or stratified the players into three regions:
  - players who have played for fours or lesser years,
  - players who have played for five or more years and who have scored less than 118 hits in previous years,
  - players who have played for five or more years and who have scored at least 118 hits in previous years.



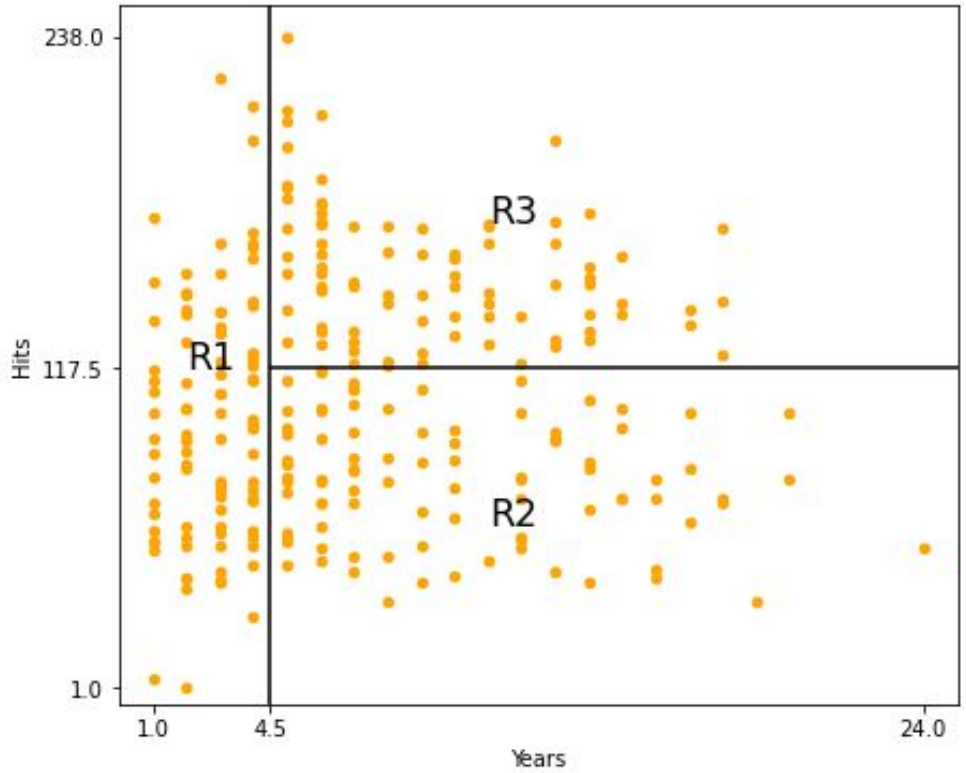
Predict Baseball Player Salary Using Tree

- We can denote these regions as
  - $R_1 = \{X|Years < 4.5\}$
  - $R_2 = \{X|Years \geq 4.5, Hits < 117.5\}$,
  - $R_3\{X|Years \geq 4.5, Hits \geq 117.5\}$.
- We can predict the salary as
  - $\$1000 \times e^{5.107} = \$165{,}174$,
  - $\$1000 \times e^{5.999} = \$402{,}834, and$
  - $\$1000 \times e^{6.740} = \$845{,}346$ respectively.



Predict Baseball Player Salary Using Tree

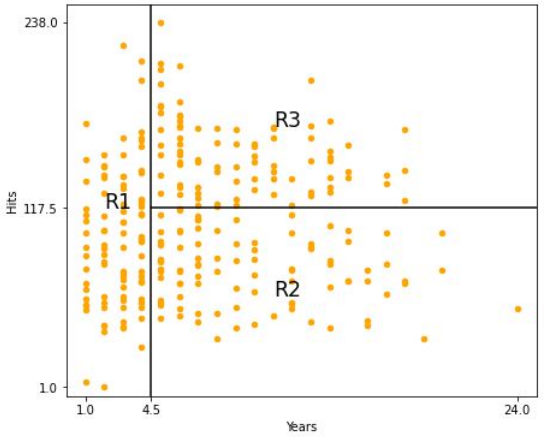
- The Figure – 4 is analogous to tree.
- We can call the regions R_1, R_2, and R_3 as terminal nodes or leaves of the tree.
- The decision trees are generally upside down.
- The leaves are at the bottom of the tree.
- The points along the tree where the predictor space is split is referred as internal nodes.
- The two internal nodes are Years < 4.5 and Hits < 117.5.
- The segments of the tree that connect the nodes are called branches.



Predict Baseball Player Salary Using Tree

- In this case study, Years is the most important factor that determine Salary.
- Players with lesser experience earn lower salaries as compared to the more experience players.
- If the player has less experience, then the number of hits he made in the previous years has a little role in his salary.
- The number of hits becomes an important factor in deciding the Salary of player if he has five or more years of experience.
- The player who made more hits last year, gets more Salary.

Process of building regression tree



- There are two steps of building a regression tree
  - Divide the predictor space. In this step, we divide the set of possible values for $X_1, X_2, \ldots, X_p$ into J distinct and non-overlapping regions $R_1, R_2, \ldots, R_J$
  - For every observation that falls into the region R_j, we make the same prediction based on the mean of the response value of training observation.
- For step 1, suppose we obtained two regions R_1 and R_2.
- If the response mean of the training observations in the first region is 10, whereas the response mean of the training observations in the second region is 20.
- For a given observation $X = x$, if $x \in R_1$, we would predict the value of 10.

Process of building regression tree

- If $x \in R_2$, we predict a value of 20.
- Now the question arises, how do we construct the region $R_1, \ldots, R_J$? We find the region $R_1, \ldots, R_J$ such the RSS given by the following formula is minimized.

$$\sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_J} \right)^2$$

- $\hat{y}_{R_J}$ is the mean response for the training observations within the jth box.

Process of building regression tree

- We cannot consider every possible partition of the feature space into J boxes.
- Hence, we take a top-down greedy approach which is known as recursive binary splitting.
- We call it top-down because it begins at the top of the tree (at which point all the observations belong to a single region), and then split the predictor space successively.
- We call it greedy because at each step of the tree building process, the best split is made at a particular step, rather than looking ahead and picking a split that would lead us to a better tree in future steps.

Process of building regression tree

- To perform the recursive binary splitting, we select the predictor X_j and the cut point s in such a way that by splitting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the greater possible reduction in RSS.
- Next, we repeat the process and search for the best predictor and best point to split the data further by minimizing the RSS within each of the resulting regions.
- However, this time, we do not split the entire predictor space.
- We only split one of the two previously identified regions.

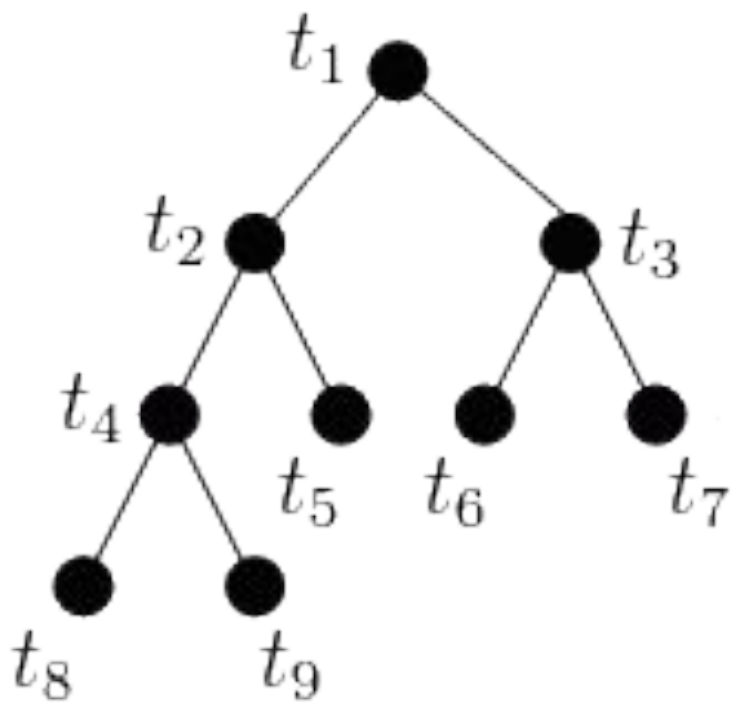
Process of building regression tree

- Thus, we get three regions now.
- We can again split one of the three regions further so that we can minimize the RSS.
- We continue the process until the stopping criterion is reached.
- The example of the stopping criterion could be that no region contains more than five observations.

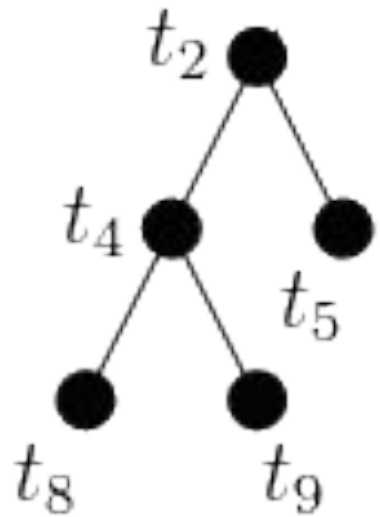
Tree Pruning

- The process given above may result into a complex tree.
- Due to which, we may get good predictions on the training set, but may get poor test set performance.
- A smaller tree with fewer split might lead to lower variance and better interpretation at the cost of a little bias.
- There are several approaches to get a smaller tree.
- One of the approaches would be to continue building the tree until the decrease in the RSS due to each split exceeds some high threshold.
- We will get a smaller tree but it is a short-sighted approach.
- A seemingly worthless split at early stage may be followed by a very good split, which can lead to a larger reduction in the RSS later on.

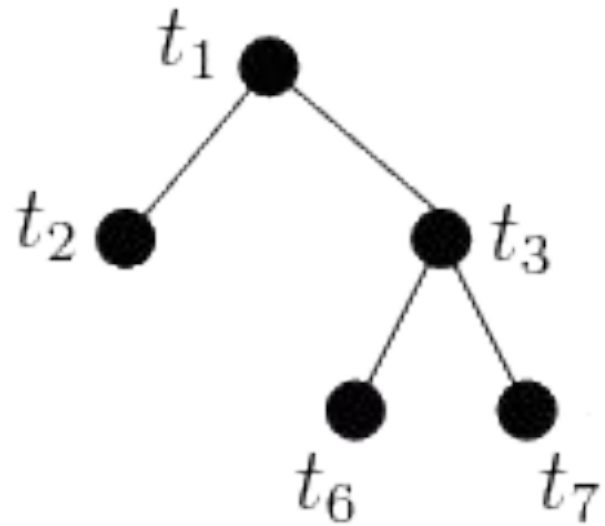
tree T



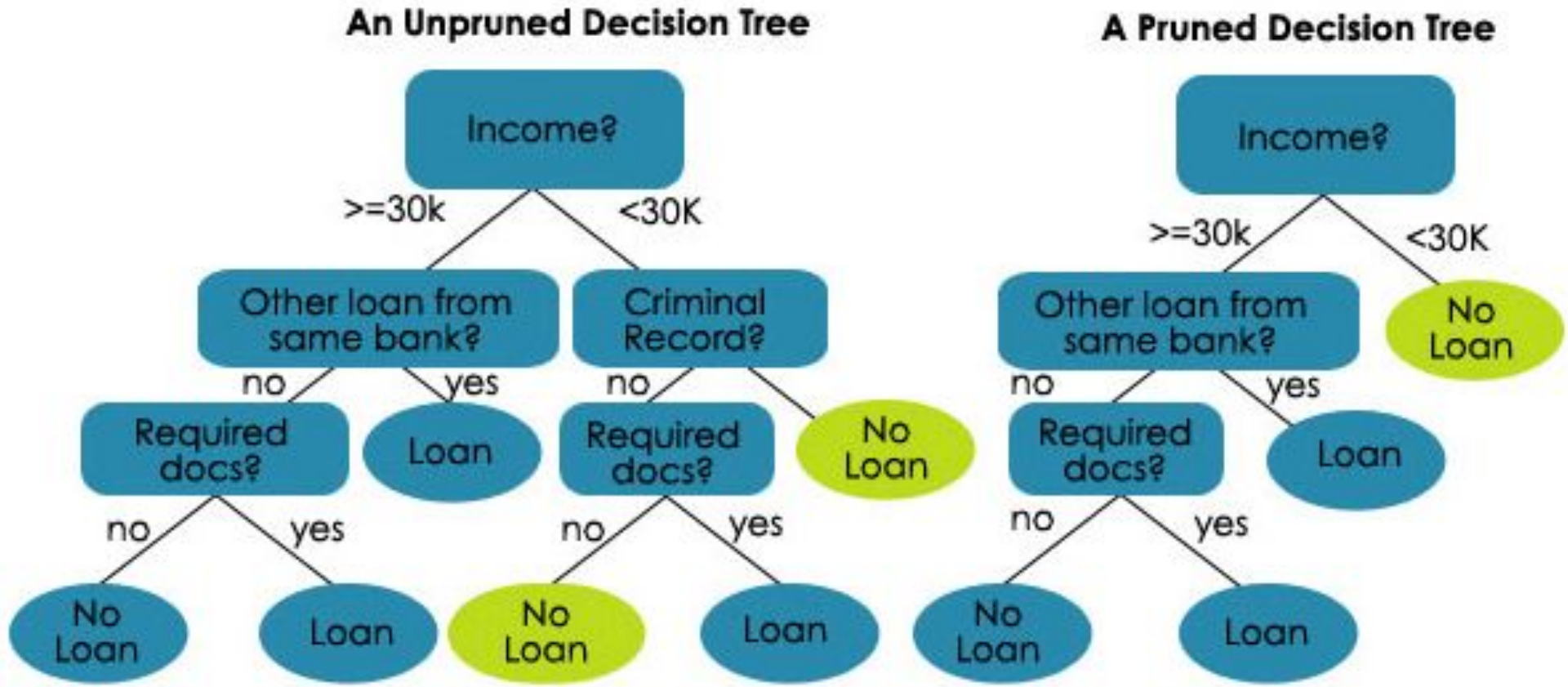
subtree T_{t_2}



subtree $T - T_{t_2}$



Tree Pruning Example



Tree Pruning

- Another viable approach would be to grow a very large tree T_0 and then **prune** it back to get a **subtree**.
- Our goal is to select a subtree that has the lowest test error rate.
- We can estimate the test error rate using cross-validation or the validation set approach.
- But there can be an extremely large number of possible subtrees that would be too cumbersome.
- Hence, we need a method to select a small set of subtrees for consideration.

Tree Pruning

- Another approach would be **cost complexity pruning** which is also known as **weakest link pruning**.
- In this approach, instead of considering every possible subtree, we consider a sequence of trees indexed by a nonnegative tuning parameter α.
- For each value of α, there is a corresponding subtree $T \subset T_0$ such that the following relation is as small as possible.

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

- In this formula $|T|$ indicates the number of terminal nodes of the tree T, R_m is the rectangle corresponding to the mth terminal node, and $\hat{y}_{R_m}$ is the predicted response that is associated with R_m.

Tree Pruning

- In this case the tuning parameter α controls a trade-off between the subtree's complexity and it's fit to the training data.
- If $\alpha = 0$, the subtree will be equal to T_0 and it just measures the training error.
- For the higher value of α, there is a penalty for having a tree with many terminal nodes.
- Hence the equation will be minimized for a smaller subtree.
- The equation above is reminiscent of the lasso where we used the similar formulation to control the complexity of a linear model.

Tree Pruning

- As we increase α from zero, the branches are pruned from the tree in a nested and predictable fashion.
- Hence, we can easily get a whole sequence of subtrees as a function of α.
- We can select the value of α using a validation set or using cross-validation.
- We then return to the full dataset and obtain the subtree corresponding to α.

Classification Tree

- As we have seen earlier that classification trees are used for qualitative response.
- In the case of classification tree, we predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.
- For results interpretation of a classification tree, we are not only interested in the class prediction corresponding to a particular terminal node region, but also in the class proportions among the training observations that fall into that region.

Classification Tree

- The classification tree is grown in the similar ways as we grow the regression tree.
- But in the case of the classification tree, we do not use RSS.
- Rather we use other measures that are based on the degree of impurity of the child nodes.
- At each node the tree will take the following steps:

Classification Tree

1. Calculate the purity of the data
2. Select a candidate split
3. Calculate the purity of the data after the split
4. Repeat for all the variables
5. Choose the variable with the greatest increase in purity or information gain.
6. Repeat for each split until some stop criteria is met

Impurity Criterion

Gini Index

$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

p_j: proportion of the samples that belongs to class c for a particular node

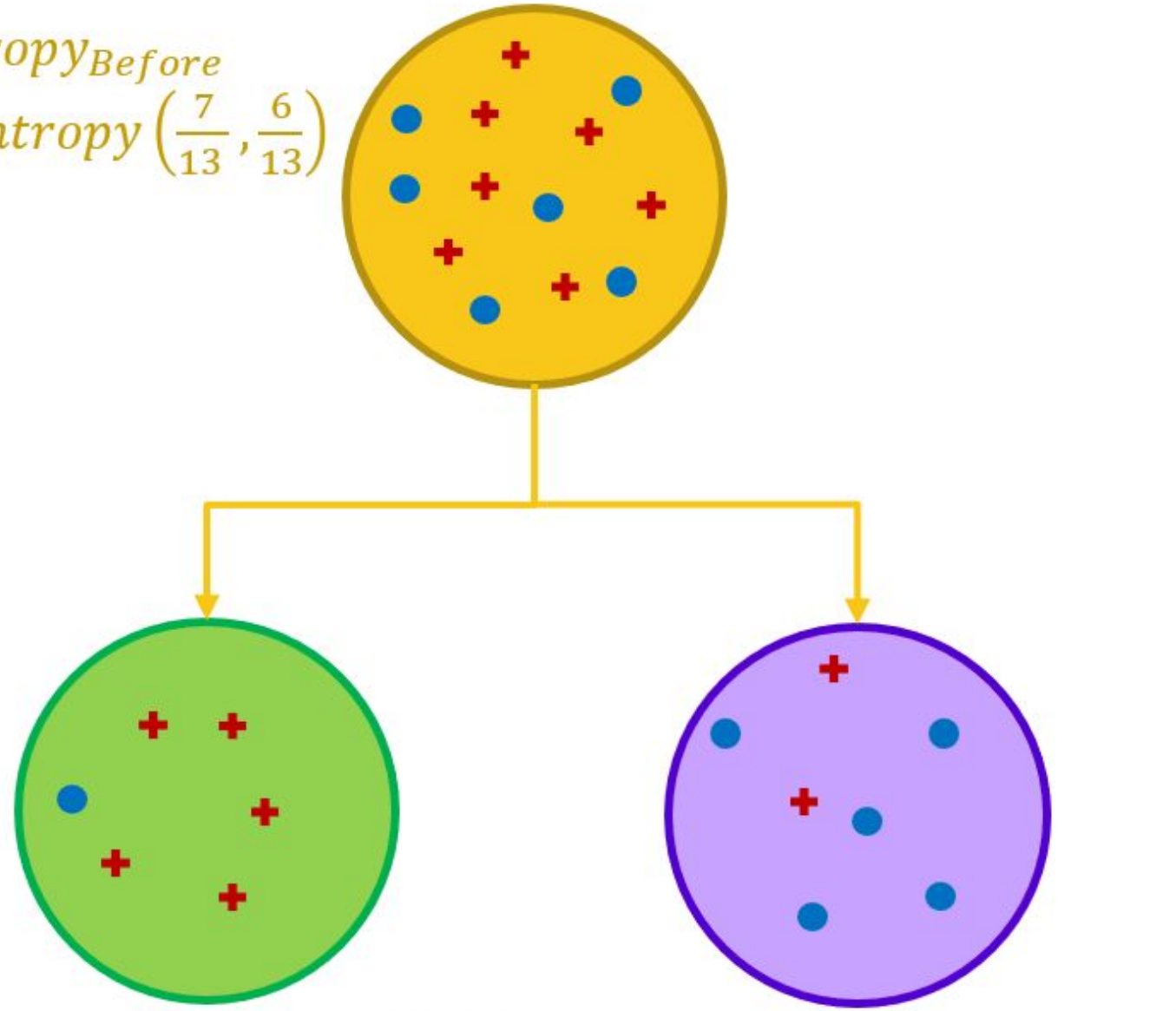
Entropy

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$

p_j: proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$). The entropy is 0 if all samples at a node belong to the same class.

$$Entropy_{Before} = Entropy\left(\frac{7}{13}, \frac{6}{13}\right)$$



$$Entropy_1 = Entropy\left(\frac{5}{6}, \frac{1}{6}\right)$$

$$w_1 = {}^{6}/_{13}$$

$$Entropy_2 = Entropy\left(\frac{2}{7}, \frac{5}{7}\right)$$

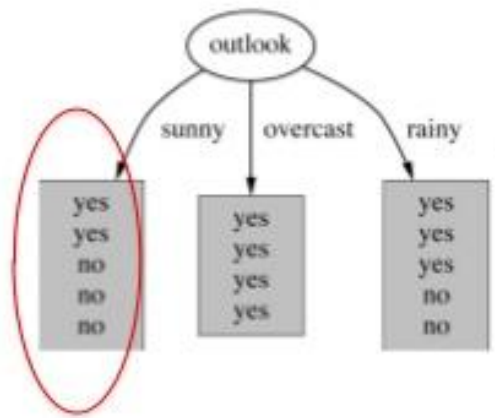
$$w_2 = {}^{7}/_{13}$$

Entropy: Outlook, sunny

- Formulae for computing the entropy:

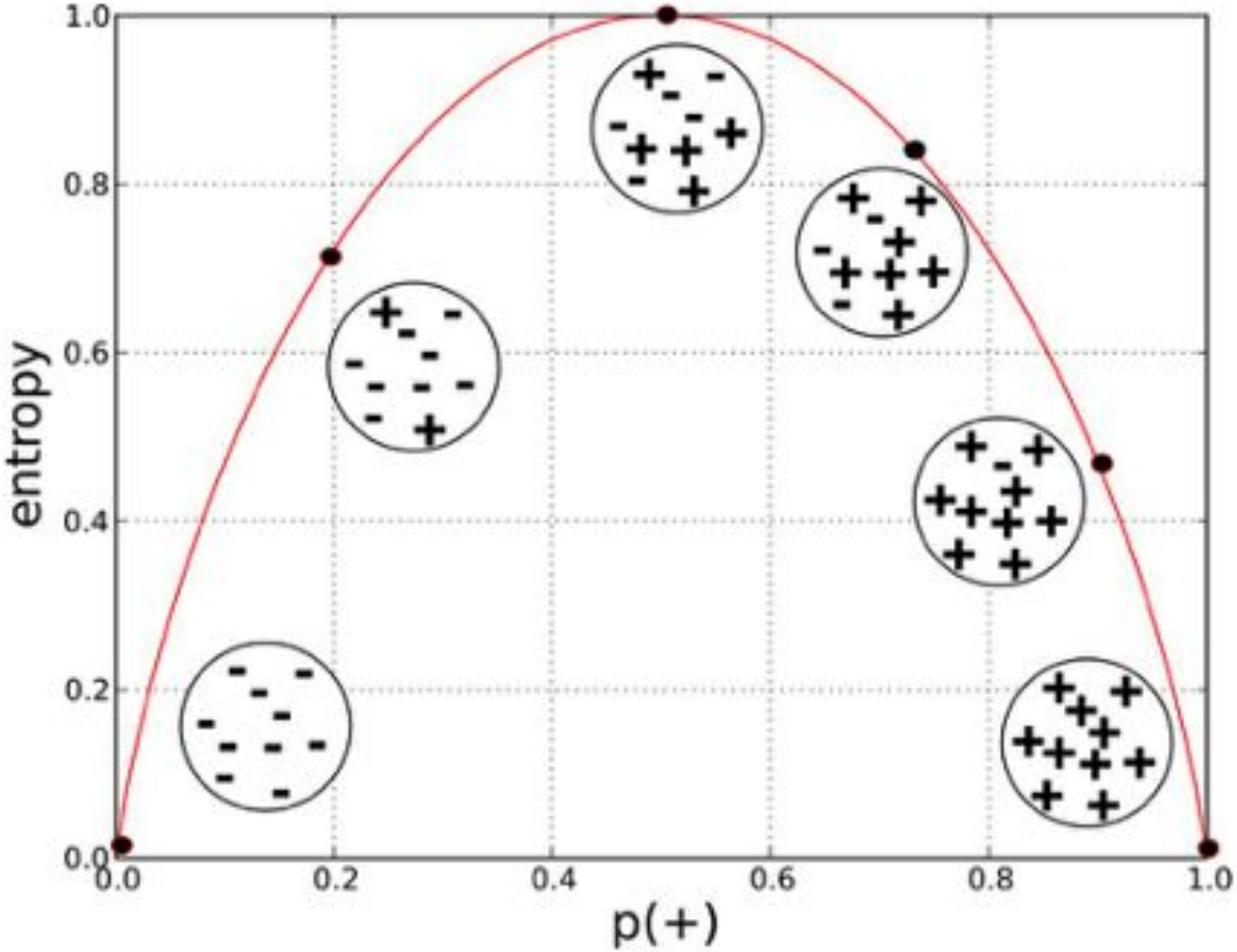
$$\text{entropy}(p_1, p_2, \ldots, p_n) = -p_1 \log p_1 - p_2 \log p_2 \ldots - p_n \log p_n$$

$$H(X) = -\sum_x p(x) \log p(x)$$



$$\text{info}([2,3]) = -2/5 \times \log 2/5 - 3/5 \times \log 3/5 = 0.971 \text{ bits},$$

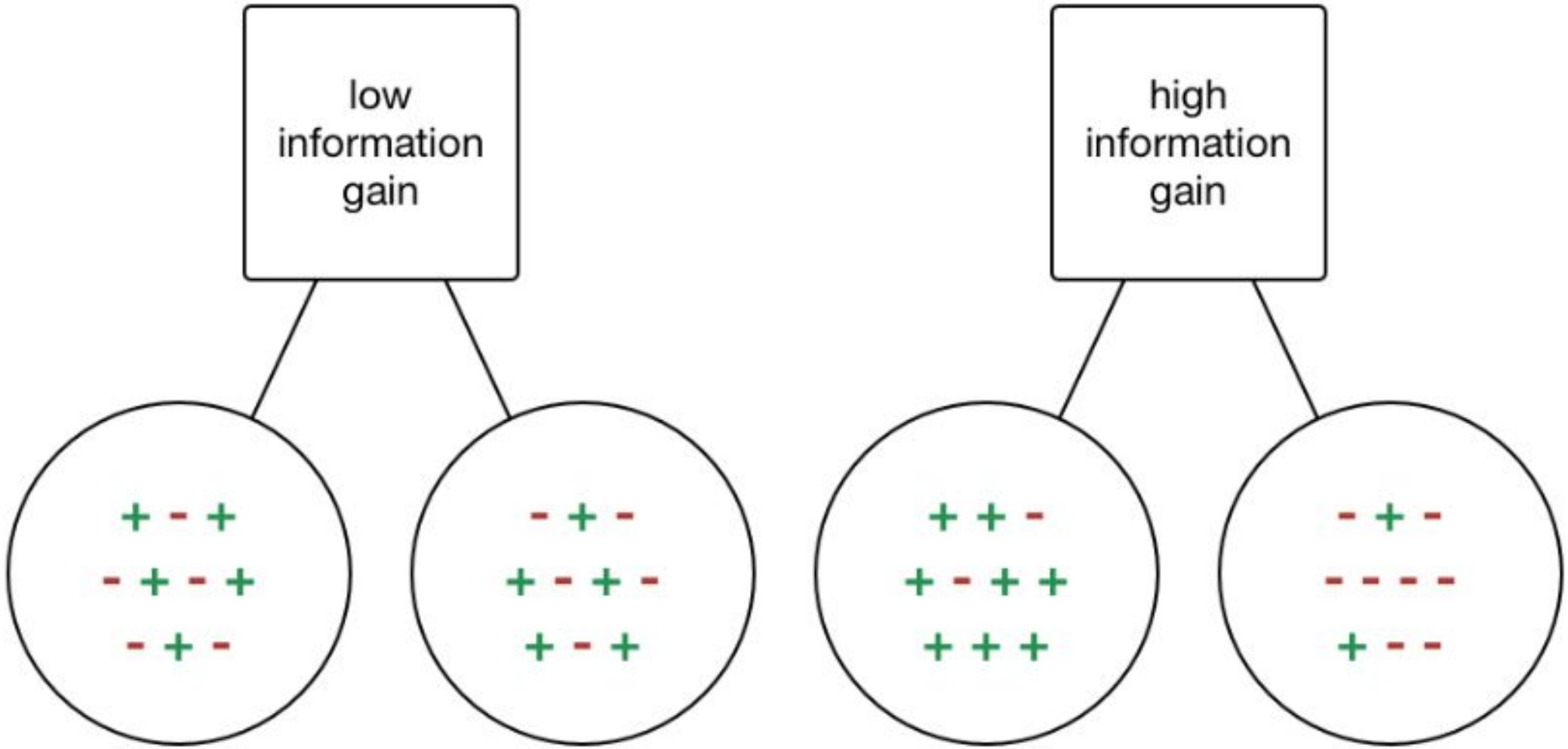
= (((-2) / 5) log2(2 / 5)) + (((-3) / 5) x log2(3 / 5)) = 0.97095059445



Information Gain

- **IG** is a statistical property that measures how well a given attribute separates the training examples according to their target classification. Constructing a decision tree is all about finding an attribute that returns the highest information gain and the smallest entropy.

$$Information\ Gain = Entropy(before) - \sum_{j=1}^{K} Entropy(j, after)$$



Impurity Measures

- Important impurity measures are

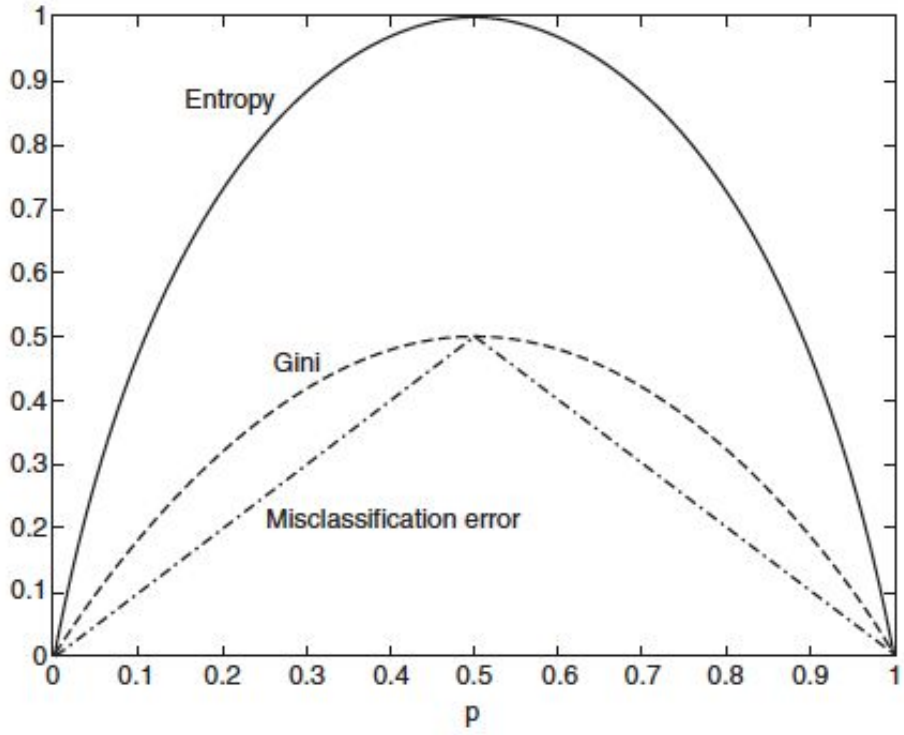
$$Entropy\ (t) = -\sum_{i=1} p(class_i|total) \log_2 p(class_i|total)$$

$$Gini\ (t) = 1 - \sum_{i=i} [p(class_i|total)]^2$$

$$\textit{Miss Classification Error}\ (t) = 1 - max[\mathrm{p}(class_i|total)]$$

Impurity Measures

- Figure – 5 compares the binary values of the impurity measures for binary classification problem.
- p refers to the fraction of records that belong to one of the two classes.
- For all the three measures, we get the maximum value at $p = 0.5$.
- The minimum value is attained when all the records are from the same class (i.e., $p = 0$ or $p = 1$)



Case Study

- For example, if our objective is to predict the likelihood of death aboard a luxury cruise ship driven by demographic features.
- In this case, if we start with 25 people, 10 of whom survived, and 15 of whom died.

Before Split	Counts
Survived (S)	10
Died (D)	15
Total (T)	25

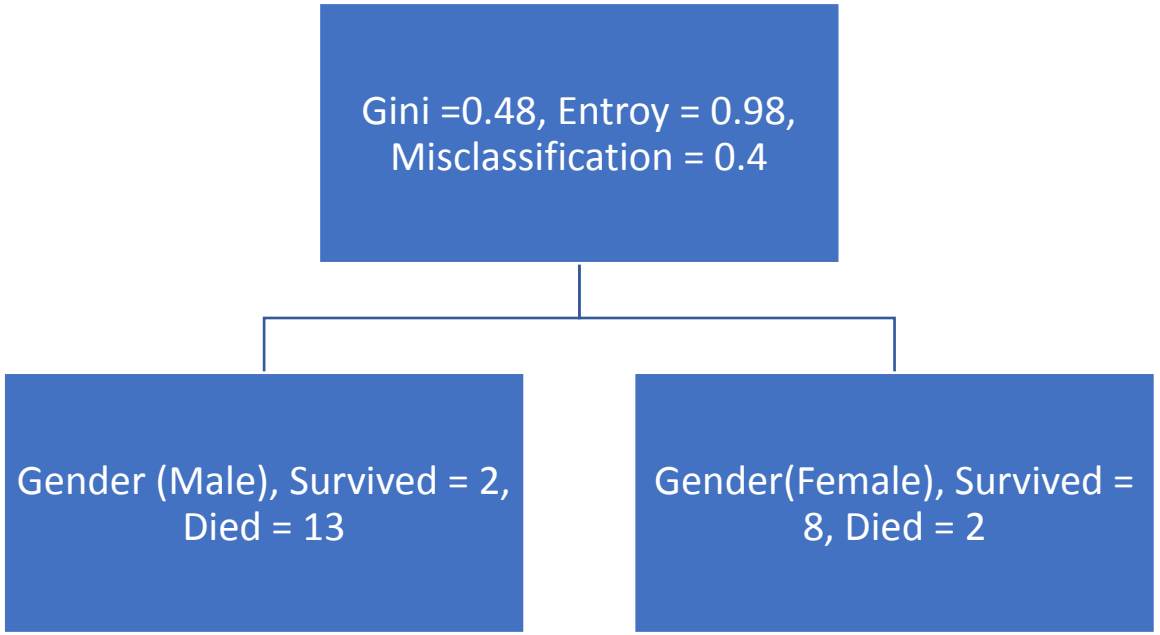
Case Study

- Now we calculate the measures

Gini		
Entropy		
Misclassification		

Case Study

- Now we consider the potential split on Gender



Case Study

- Now we can calculate the impurity measures for both male and female

Measure	Male Node	Female Node
Gini		
Entropy		
Misclassification		

Case Study

- We notice that Male node has lower degree of impurity than the Female node.
- This is expected because Male node is purer than the Female node.
- Now we calculate overall index of the gender

$$Gini(Gender) = Gini(M)\left(\frac{M}{M+F}\right) + Gini(F)\left(\frac{F}{M+F}\right)$$
$$= 0.23\left(\frac{15}{25}\right) + 0.32\left(\frac{10}{25}\right) = 0.27$$

$$Entropy(Gender) = Entropy(M)\left(\frac{M}{M+F}\right) + Entropy(F)\left(\frac{F}{M+F}\right)$$
$$= 0.57\left(\frac{15}{25}\right) + 0.72\left(\frac{10}{25}\right) = 0.63$$

Case Study

- Therefore, we can calculate the information gain by splitting on the Gender feature as:
- Entropy

$$IG = Entropy(Parent) - Entropy(Gender) = 0.97 - 0.63 = 0.34$$

- Gini

$$IG = Gini(Parent) - Gini(Gender) = 0.48 - 0.27 = 0.21$$

- Now we follow the procedure for 3 potential splits
  - Gender (male or female)
  - Number of siblings on board (0 or 1+)
  - Class (first and second versus third)

Case Study

Gender	Male	Female
Survived	2	8
Died	13	2
IG(Entropy)	0.34	
IG(Gini)	0.21	

Sibling	0	>= 1
Survived	5	5
Died	7	8
IG(Entropy)	0.00	
IG(Gini)	0.00	

Class	1,2	3
Survived	7	3
Died	5	10
IG(Entropy)	0.06	
IG(Gini)	0.09	

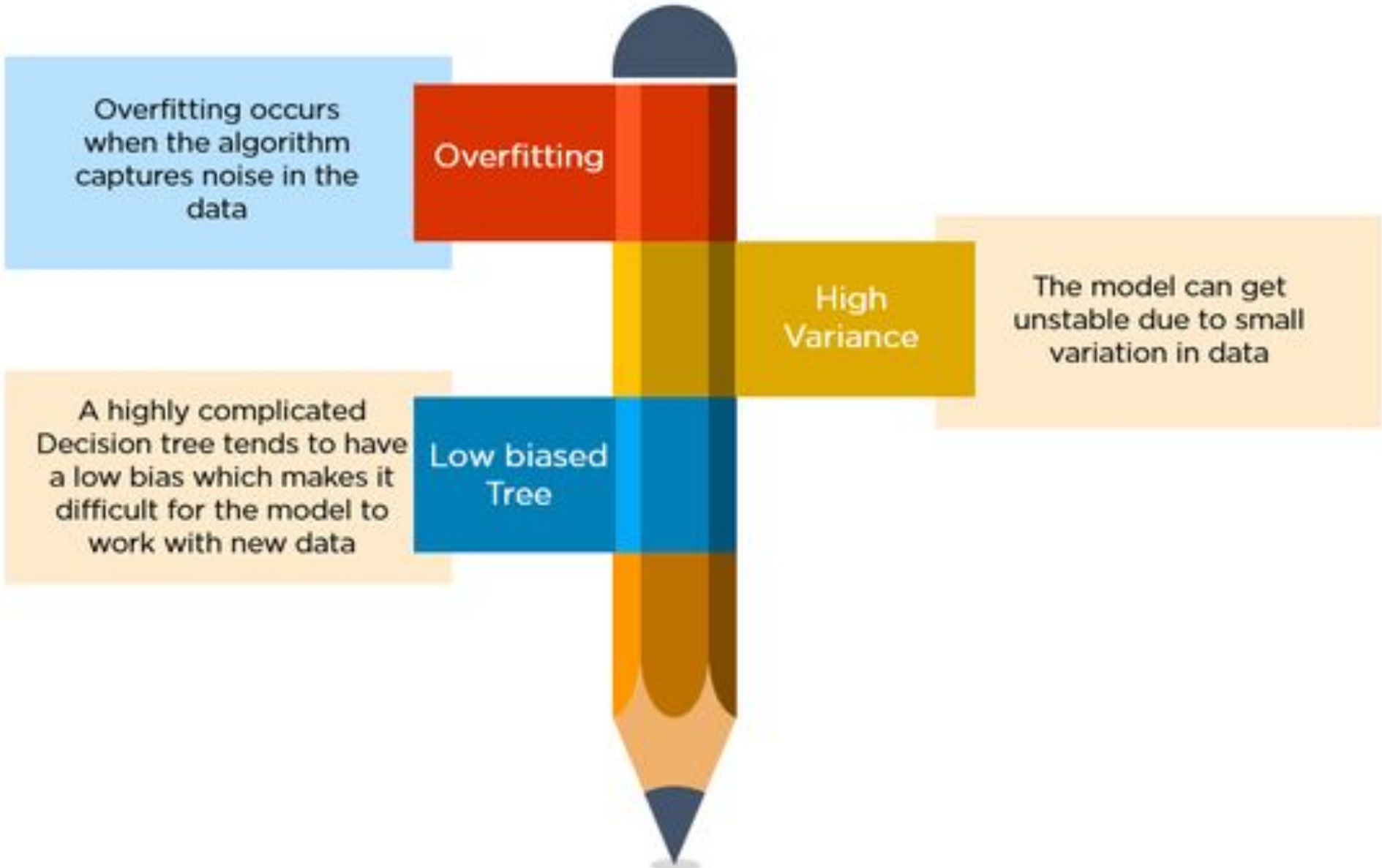
Advantages of Trees

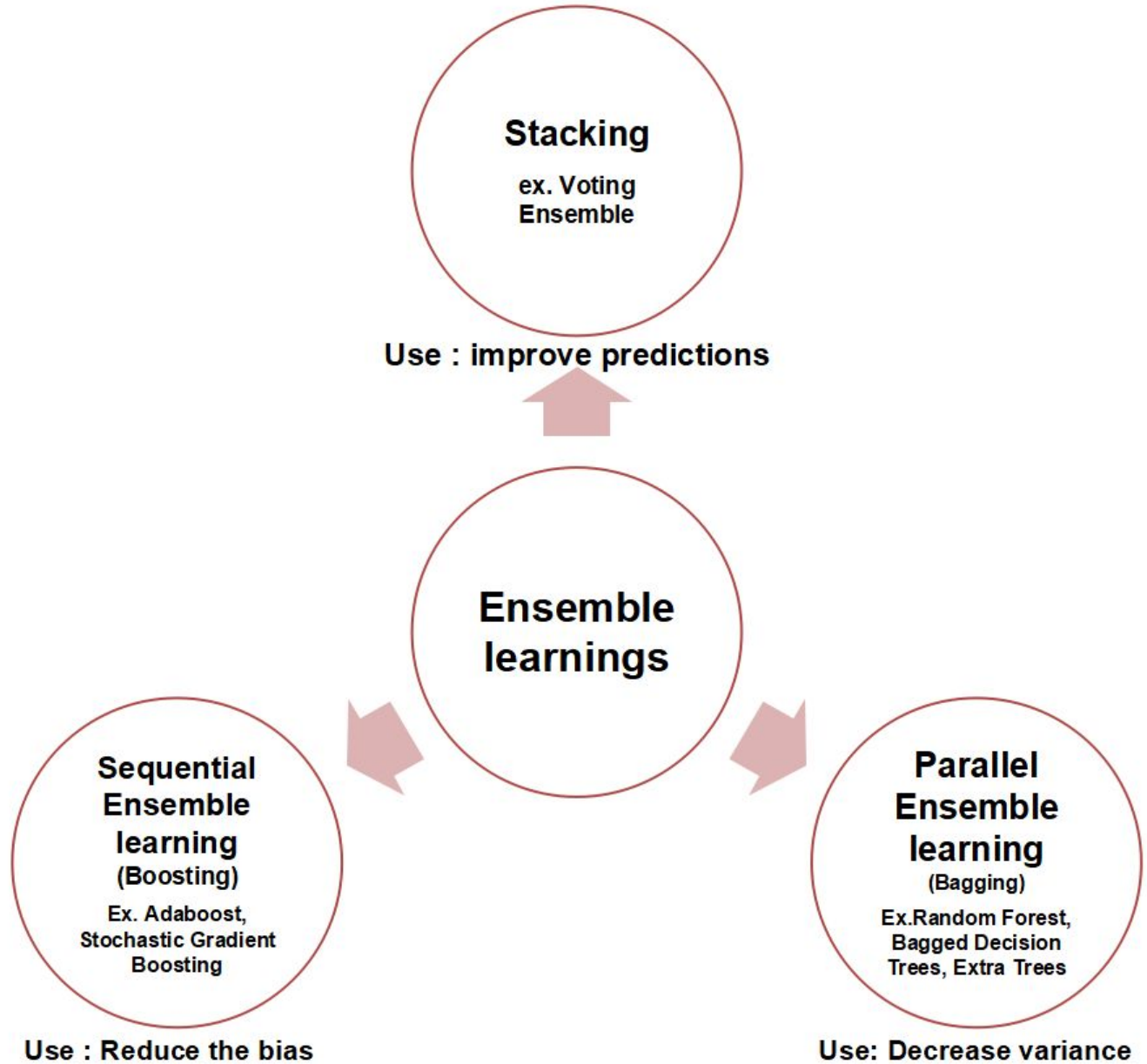
- Easy to explain. Infect they are easier than the linear regression
- Some experts believe that decision trees closely imitate the human decision-making process than the regression and classification approaches
- We can represent the decision tree graphically. Hence, they are easy to interpret for non-experts
- Decision tree can easily handle qualitative predictors without creating dummy variables

Disadvantages of Trees

- Low predictive accuracy. They have lower predictive accuracy than the linear regression and classification approaches
- Decision trees are not very robust. A small change in the data can cause a large change in the final estimated tree.

We can improve the predictive accuracy of the decision trees using bagging, boosting, and random trees.


Overfitting occurs when the algorithm captures noise in the data
Overfitting
High Variance
The model can get unstable due to small variation in data
A highly complicated Decision tree tends to have a low bias which makes it difficult for the model to work with new data
Low biased Tree


Stacking
ex. Voting Ensemble
Use : improve predictions
Ensemble learnings
Sequential Ensemble learning
(Boosting)
Ex. Adaboost, Stochastic Gradient Boosting
Use : Reduce the bias
Parallel Ensemble learning
(Bagging)
Ex.Random Forest, Bagged Decision Trees, Extra Trees
Use: Decrease variance

Bootstrap sampling

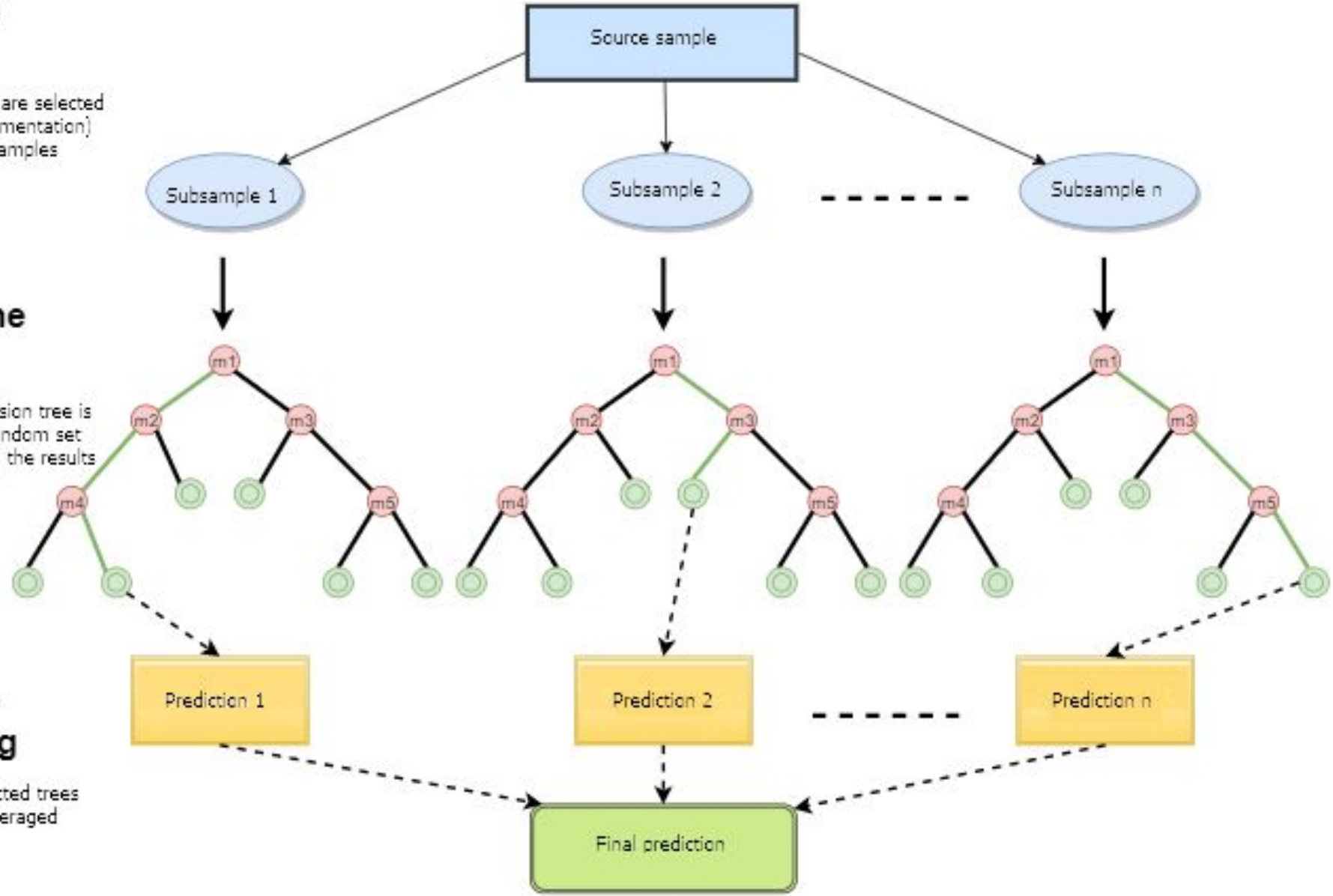
r (percentage) examples are selected (0.63 in classical implementation) in n random subsamples

Building the models

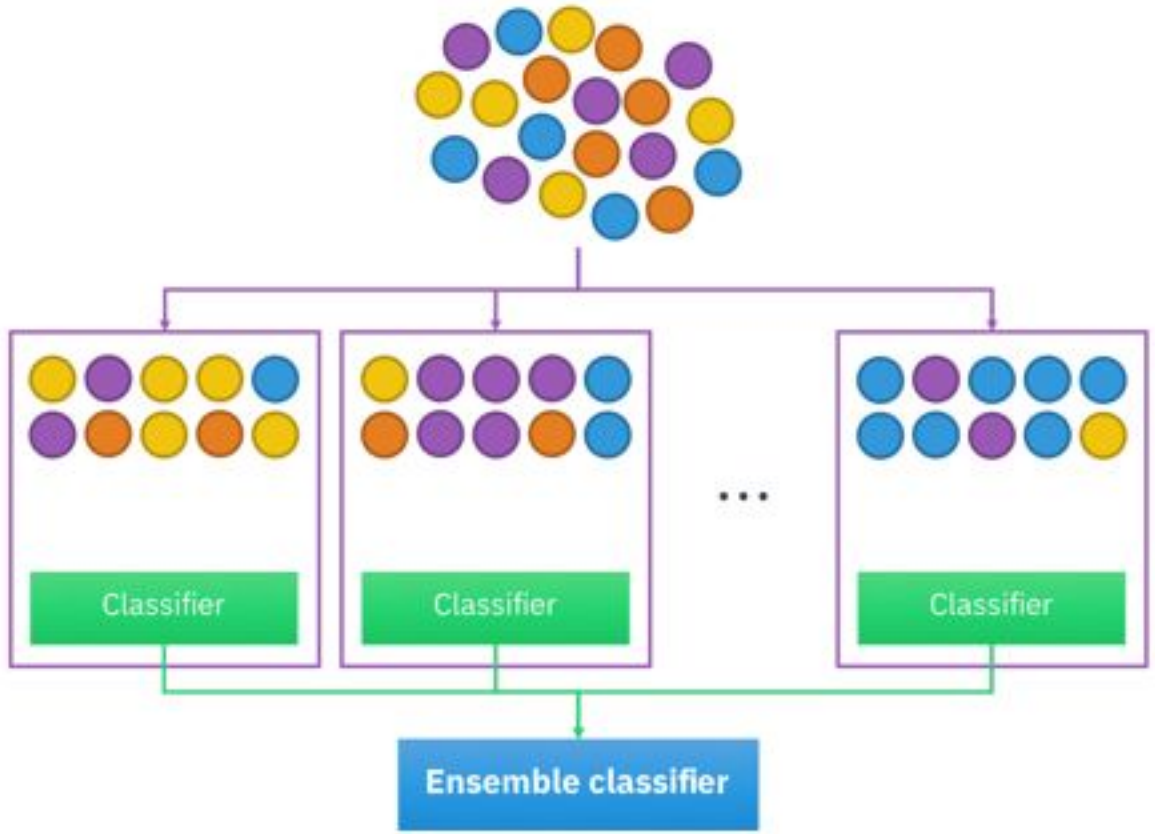
for each subsample, a decision tree is constructed based on a random set of m features (covariants), the results fall into leaves

Bootstrap aggregating

results from all constructed trees are gathered and averaged



BAGGING LEARNING PROCEDURE



$$f(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$$

Weak Learners

Generates Bootstrapping Sets

Bagging

- In the previous chapter we discussed about the bootstrap method which is an extremely powerful idea.
- We use it in the situations where it is hard or impossible to directly compute the standard deviation of a quantity of interest.
- In this section, we will see how we can use the bootstrap method to improve the decision trees

Bagging

- The decision tree, that we discussed so far suffer from **high variance**.
- This means that it the if split the training data into two parts at random, and fit a decision tree to both the halves, every time we will get the different results.
- Where, we will get the similar results if model is applied to distinct data set.
- We can use **bootstrap aggregation** or **bagging** to reduce the variance of a statistical learning method.

Bagging

- From statistics, we learned that if we have n different random samples $Z_1, \ldots, Z_n$, each with variance σ^2, the variance of the mean $\bar{Z}$ is σ^2/n.
- We can also say that by averaging a set of observations, we can reduce the variance.
- This is a natural way to reduce the variance and increase the prediction accuracy.
- We can take many training sets from the population, build a separate prediction method using each training set, and take a mean of the resulting predictions.
- Hence, we can obtain a single low-variance machine learning model, by first calculating $\hat{f}^1(x), \hat{f}^2(x), \ldots, \hat{f}^B(x)$ using B separate training sets, and finally averaging them using

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$$

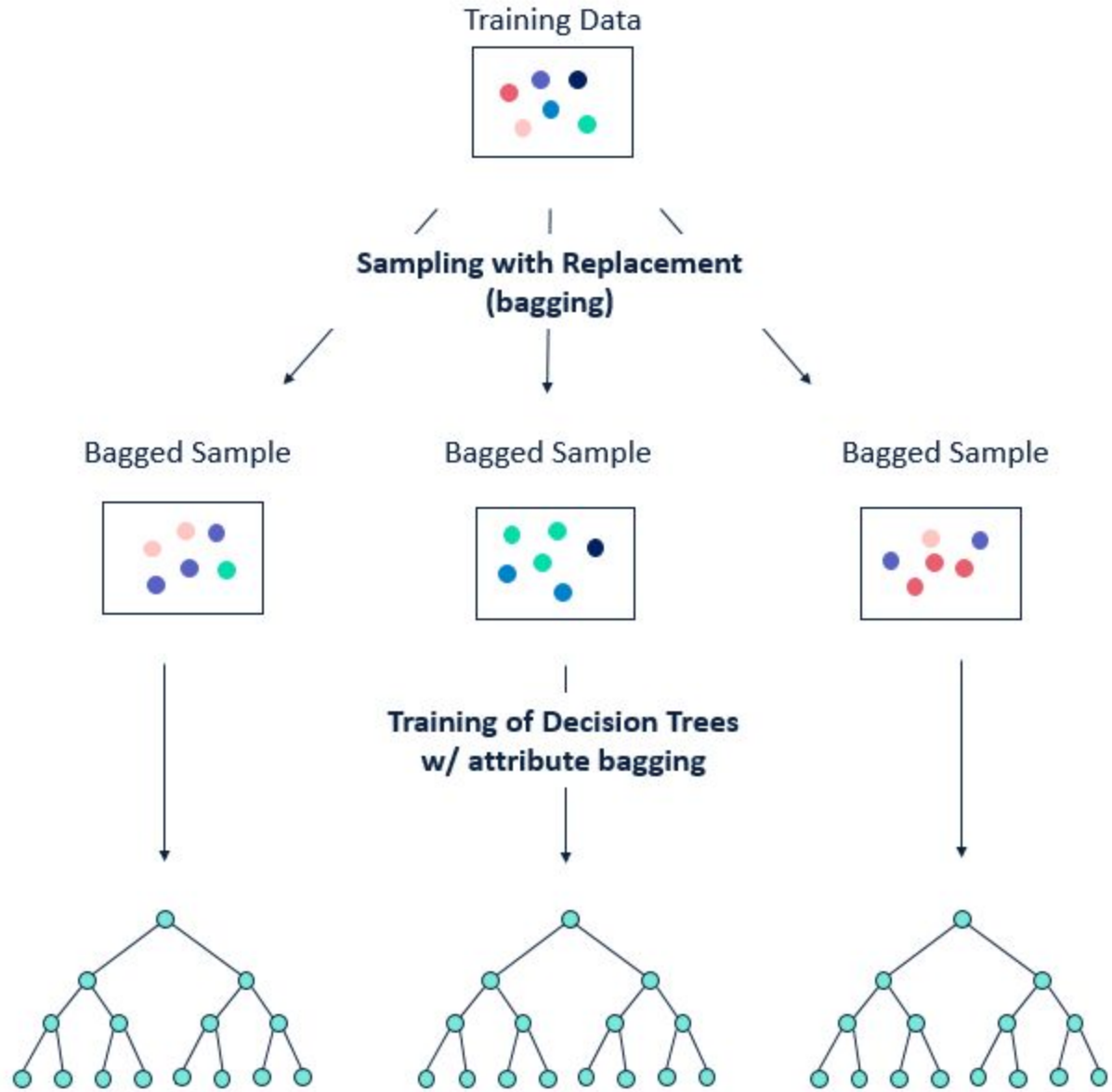
Bagging

- In practice, we cannot access multiple training set.
- In this case, we use bootstrap method where we take repeated samples (with replacement) from a single training set.
- Therefore, we get B different bootstrapped training data sets.
- Now we train our model on bth bootstrapped training set to get $\hat{f}^{*b}(x)$. Finally, we average all the predictions to get

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

Bagging

- This is called **bagging**.
- The Bagging has demonstrated impressive improvements in accuracy by combining hundreds or even thousands of trees into a single procedure.
- In order to implement bagging to predict a qualitative outcome Y, we can use several approaches.
- One of the most common approach is to take a **majority vote**, whereas the overall prediction is the most commonly occurring class among B predictors.


Training Data
Sampling with Replacement
(bagging)
Bagged Sample
Bagged Sample
Bagged Sample
Training of Decision Trees
w/ attribute bagging
Random Forest

Random Forest

- Random Forests provide an additional improvement over bagging.
- Random forests algorithm uses bagging to create a large number of trees but uses an extra trick.
- At each node of a particular tree, the random forests randomly select m number of the predictor out of the full set of p predictors, that it would consider for that split.
- Typically, it selects the m number of predictors such that $m \approx \sqrt{p}$.

Random Forest

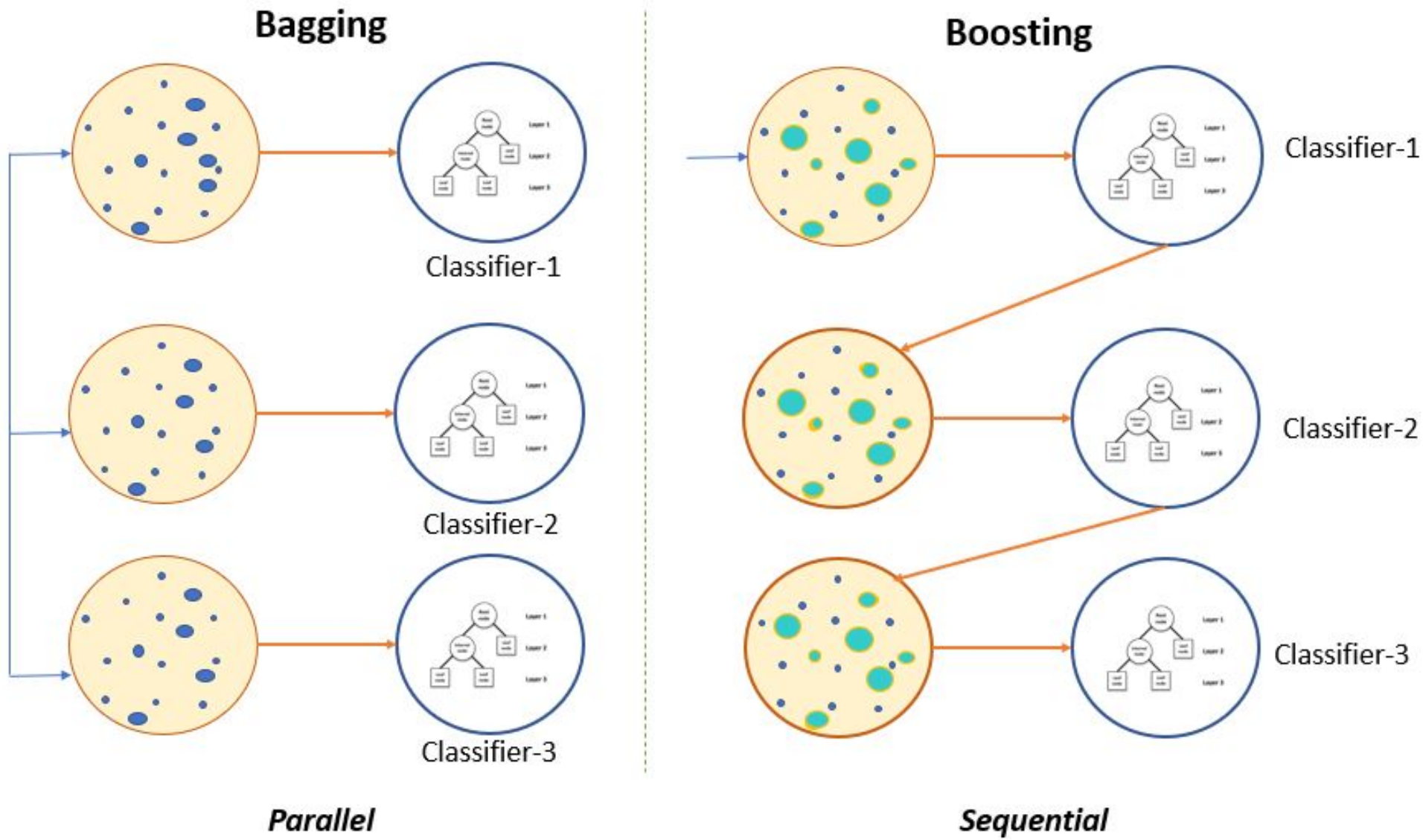
- Thus, the random forest even does not consider majority of available predictors.
- It may seem counterintuitive, but is has a clever rationale.
- Suppose there is a very strong predictor in the data set, along with a number of other moderately strong predictors.
- In this case, in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split.
- Hence, all the bagged trees will be similar to each other.
- Due to this, the predictions from the bagged trees will be highly correlated.
- The predictions from highly correlated quantities will not lead to a significant reduction in variance as compared to uncorrelated quantities.
- We can also say that in this setting, bagging will not lead to a significant reduction in variance over a single tree.

Random Forest

- Random forest overcome this problem by considering only a subset of the predictors for each split.
- In the case of random forest, on $(p - m)/p$ of the splits will not even consider the strong predictors and hence other predictors will have more chance.
- This process is also known as **decorrelating** the trees.
- This process results in reducing the variance of the average of the resulting trees and hence more reliable.

Random Forest

- The main difference between bagging and random forests is the choice of the predictor subset size m.
- If $m = p$, the random forest will amount to bagging.
- If we have a large number of correlated predictors, we should use a small value of m in building the random forest.


Bagging
Classifier-1
Classifier-2
Classifier-3
Parallel
Boosting
Classifier-1
Classifier-2
Classifier-3
Sequential

Boosting

- With bagging, the individual models are trained in parallel.
- **Boosting** also trains many individual models, but builds them sequentially.
- Each additional model seeks to correct the mistakes of the previously grown tree by using the information from the previously grown tree.
- Boosting does not involve bootstrap sampling but each tree is fit on a modified version of the original data set.

Boosting

- Just like bagging, boosting can be applied to any supervised machine learning algorithm.
- However, boosting is most beneficial when using weak learners as the sub-models.
- By weak learner, we don’t mean someone who keeps failing their driving test; we mean a model that only does a little better at making predictions than a random guess.
- For example, a binary classifier that can achieve slightly better than 50 percent accuracy.
- For this reason, boosting has been traditionally applied to shallow decision trees.
- By shallow, we mean a decision tree that doesn’t have many levels of depth, or may have only a single split.

Thanks

Samatrix Consulting Pvt Ltd